Noname manuscript No.
(will be inserted by the editor)

The State of the SBOM Tools Ecosystem and their Prevalence in OSS: A Comparative Analysis of SPDX and CycloneDX

Abdul Ali Bangash · ~~Arshdeep Singh~~
Tongxu Ge · Zhimin Zhao · ~~Zitao Wang~~
~~Ankit Hans~~ Bram Adams

Received: date / Accepted: date

Abstract A Software Bill of Materials (SBOM) is an essential component of modern software development that is often shipped with a software release. An SBOM provides transparency and accountability of a release by documenting the metadata of its components and their dependencies (e.g., version numbers). However, to generate, edit, visualize, and integrate an SBOM into the software process, one requires software tools (often known as SBOM tools). In this paper, we explore the open-source tool ecosystem surrounding the two main SBOM ~~formats~~standards, SPDX and CycloneDX, to offer insights into their usage, strengths, weaknesses, and potential areas for improvement. Firstly, our study conducts a qualitative comparison of the use cases for 108 open-source and 62 proprietary SBOM tools, identifying areas where open-source tools of each ~~format~~ standard may require enhancements in their features. Secondly, a comparison of the health metrics for each ~~format~~standard's tool ecosystem is made (171 CycloneDX tools vs. 470 SPDX tools). Additionally, to provide concrete insights about the challenges and potential development areas of each ~~format~~standard, the study quantitatively compares 36,990 issues reported to the open-source tool ecosystem of CycloneDX and SPDX. Finally, we investigate the characteristics of the top 250 open-source projects using CycloneDX tools and compare their health metrics with the top 250 open-source projects that use SPDX tools. The findings reveal that projects utilizing CycloneDX tools have higher developer engagement and better project health compared to the SPDX tools, suggesting that CycloneDX tools are more appealing to healthier and active projects. This research provides valuable insights for software developers, open-source contributors, and SBOM practitioners, guiding decision-making processes to improve the tool ecosystem, highlighting areas for enhancement, and offering directions for further development.

Abdul Ali Bangash
Software Analysis and Intelligence Lab (SAIL)
Queen's University, Kingston, ON, Canada
E-mail: abdulali.b@queensu.ca

Bram Adams
Lab on Maintenance, Construction and Intelligence of Software (MCIS)
Queen's University, Kingston, ON, Canada
E-mail: bram.adams@queensu.ca

Keywords OSS, Software Ecosystem, SPDX, CycloneDx, Software Bill of Materials

1 Introduction

Modern software applications rely heavily on third-party dependencies, forming the foundation of their software supply chain (Cox, 2019). However, the increasing number of dependencies pose significant challenges in terms of security and reliability (Gkortzis et al., 2021). Malicious actors are known to exploit vulnerabilities in less secure dependencies, ultimately tainting the supply chain and facilitating the breach of a high-value application that uses a compromised dependency (Ladisa et al., 2022). To address such vulnerabilities, it is crucial to have up-to-date information about all third-party dependencies in the supply chain of an application.

In the past, the absence of proper standards for managing such information has resulted in serious consequences. Notably, in 2023, OpenAI temporarily suspended ChatGPT due to a vulnerability originating from the Redis library (Sanfilippo, 2023), exposing ChatGPT's user-information (Brudo, 2023). Similarly, the SolarWinds hack in 2020 compromised over 30,000 organizations across 190 countries, including the U.S. federal agencies, highlighting the need for robust supply chain security (Brudo, 2023). Moreover, the Log4Shell vulnerability in 2021 exploited the widely used Log4j library, necessitating prompt actions to protect systems and data (Berger, 2023).

To mitigate the risk of such software supply chain attacks, in 2021, the US government issued an executive order mandating the adoption of Software Bill of Materials (SBOMs) on companies who want to conduct business with the US (Remaley and Remaley, 2021), an endeavour followed later amongst others, [redacted] the European Union, United Kingdom, Japan, and Germany (Worthington, 2023). To understand the readiness of SBOMs in the software development community, in 2022, the Linux Foundation carried out an extensive survey on 412 organizations from around the world (Hendrick, 2022). Their survey showed that more than 80% of worldwide organizations are aware of the US executive order and 76% of organizations are considering adopting the notion of SBOM as a consequence of this executive order. Overall, 47% of organizations want scalable vulnerability reporting and 45% see SBOMs as a key method to secure the software supply chain. They also found that 52% organizations are addressing SBOMs in atleast a few areas of their business; 23% are even addressing them across nearly all areas of their business or have standard practices that include the use of SBOMs. With the ever increasing understanding of SBOMs in general public, customers have started to require SBOMs from the developers of software products they use (Hendrick, 2022).

The concept of SBOM came from the notion of *bill of materials* in other engineering disciplines (van der Storm, 2007). Essentially, a SBOM is a precise specification of all "ingredients" of a given (software) product, ranging from the names and revision numbers of all source code and other files shipped with a given software release, to the names and version numbers of all third-party dependencies the product relies on, or even the licenses of those dependencies. The specific information included in an SBOM relies on the SBOM standard being used. The U.S. National Telecommunications and Information Administration (NTIA)'s gen-

eral standard of SBOM outlines which data fields must be included in an SBOM file, how SBOMs should support automation, and which practices and processes should be employed when creating, distributing, and using SBOMs (Telecommunications and , NTIA). According to NTIA (Balliu et al., 2023), three existing ~~formats~~ standards meet their requirements: CycloneDX OWASP (2023a), Software Package Data Exchange (SPDX) (SPDX, 2023c), and Software Identification (SWID) (Waltermire et al., 2016).

~~The SBOM formats~~ The SBOM standards provide structured guidelines for the data representation of software components, [redacted] (Bi et al., 2023). [redacted] the mere existence of the concept of SBOM or its ~~format~~ standard has no value without the availability of appropriate tools that support its [redacted] For instance, one must have SBOM tools to generate, visualize, and manage an SBOM of a software project. The build-supporting SBOM tools help extract key information about a software product release (such as the source code files and dependencies) during its build process. Similarly, inspection-supporting SBOM tools can be used to inspect compliance between the SBOM ~~format~~ standard and the release contents of a product, ensuring that all the components meet licensing and security requirements. This, in turn, requires more SBOM tools and libraries that can parse SBOM specifications across various programming languages.

However, in the pursuit of employing SBOM tools, software development teams encounter various problems, such as a lack of ~~format~~ standard generalization and tool maturity, as neither of the two competing SBOM standards (SPDX and CycloneDX) fully meet current market needs for extensibility across software projects (Xia et al., 2023a). [redacted] Lastly, there is generally a limited availability, usability, and integration support for SBOM tools (Xia et al., 2023a).

Xia et al. (2023a) in their comprehensive survey on SBOM practitioners (17 interviewees and 65 survey respondents) found that current tools need to be more reliable, user-friendly, and compliant with the standard ~~formats~~standards. They found that practitioners currently adopt SBOM tools in a limited capacity due to a lack of understanding about these tools, including their technical aspects, use cases, resource costs, drawbacks, and benefits. They also found that the effectiveness and sustainability of SBOM tools are crucial for ensuring the security and compliance of software supply chains. [redacted]

Understanding the properties of current SBOM tools, such as their supported ~~formats~~ standards, use cases, their health, issues, and demand, would help SBOM practitioners improve the SBOM ecosystem. Hence, in this paper, to assist SBOM practitioners and the research community in understanding the strengths and addressing the limitations of current SBOM tools, we study the tool ecosystem of two major SBOM ~~formats~~standards: SPDX and CycloneDX. We investigate the following research questions:

- RQ1: Which usecases do the current SBOM tools support?
- RQ2: What is the project-health of current OSS SBOM tools?

– RQ3: What are the common issues reported in the OSS SBOM tools?
– RQ4: What is the project-health of Github projects that use SBOM tools?

In **RQ1**, we manually investigate which SBOM use-cases the current open-source and proprietary SBOM tools support. We find that proprietary tools are richer than OSS SBOM tools when it comes to the number of ~~usecases~~ use cases they can support. Furthermore, we identify the use cases that OSS tools lack support for. Finally, we highlight the different strengths and weaknesses of each of the ~~formats~~standards' (SPDX and CycloneDX) tools.

In **RQ2**, to investigate the overall health of open-source SPDX and CycloneDX Github tools, we employ the CHAOSS metrics (OSS health analytics metrics) (C. P. L. Foundation, accessed May 27, 2024). We find that CycloneDX tools surpass SPDX tools in terms of contributor activity, number of commits, and issue resolution times. This information highlights the ecosystem of tools that is widely adopted and actively maintained, and also the areas that require the attention of developers.

In **RQ3**, to understand the potential areas of improvement in the tools of each ~~format~~standard, we extract and investigate the Github issues of the SPDX and CycloneDX SBOM tools. We manually group the most dominant issues into categories and analyzed their prevalence in both ~~formats~~standards, SPDX and CycloneDX. We find that even though CycloneDX tools have more closed issues than SPDX, their issue resolution time is also faster than SPDX.

In **RQ4**, to investigate the health of projects that use SBOM tools and to understand their choice and preference in regards to an SBOM tool, we study open-source Github projects that configure SBOM tools to automate the creation of SBOM files in their code or build release. We find that CycloneDX-based projects generally demonstrate better health metrics, including higher numbers of stars, watchers, releases, pull requests, and contributors. Additionally, projects with higher number of commits tend to have a higher adoption of SBOM tools that provide the reporting capability, while projects with lesser commits tend to prefer SBOM tools that provide SBOM edit support.

Our empirical study is valuable because it provides a comprehensive overview of the two major ecosystems of SBOM tools, their suitability for various use cases, and prevalent issues, enabling practitioners to make informed tooling decisions for SBOM management. It reveals that CycloneDX tools tend to have higher developer engagement, aiding practitioners in understanding the community dynamics of these ~~formats~~standards. Furthermore, it provides valuable insights into the usage of the SPDX and CycloneDX tool ecosystems in real-world software projects, highlighting the critical need for a robust SBOM tools ecosystem.

2 Background and Related Work

In this section, we explain what an SBOM is, what is an SBOM specification, and which ~~are the standard SBOM formats~~SBOM standards. We then explain how these concepts are related to our work and what kind of relevant studies have been performed in the literature.

2.1 What is an SBOM?

The concept of a bill of materials (BOM) originated from the manufacturing industry's practice of creating a document that records and helps to track and manage components used in the production of physical goods (Jiao et al., 2000). Drawing inspiration from this traditional notion of BOM, the concept of software bill of materials (SBOM) was first formalized in 2007 by Storm et al. (2007). An SBOM is a structured document that records the detailed information about the components and dependencies comprising a software application or system (Muirí, 2019). It keeps the names, versions, and sources of third-party libraries, frameworks, modules, and other software elements used in the development process. SBOM serves as a comprehensive record of the software supply chain, enabling organizations to uniquely and unambiguously identify components of a software and their relationships, to understand and manage the security, licensing, vulnerability, and compliance aspects of their software assets.

2.2 What is an SBOM specification?

An SBOM specification, often developed by a regulation authority, outlines the rules and information that should be included in an SBOM to ensure accurate identification and tracking of software components and dependencies. The SBOM specification typically includes details on which software assets should be described, organized, and documented within the bill of materials. An example of an SBOM specification is the U.S. NTIA's SBOM specification, which outlines the data fields that must be included in an SBOM file, how SBOMs should support automation, and which practices and processes should be employed when creating, distributing, and using SBOMs (Telecommunications and , NTIA).

2.3 Which SBOM ~~formats~~ standards are the standard for the NTIA's SBOM specification?

SBOM ~~formats~~ standards comply with and follow a specific SBOM specification to provide a structured way to document and convey information about software components. The NTIA recognizes three main SBOM ~~formats~~ standards that follow their SBOM specification: SPDX, SWID, and CycloneDX (Telecommunications and , NTIA). Each ~~format~~ standard has its unique focus and strengths, catering to different stages of the software development lifecycle and varying user needs. SPDX ~~format~~ standard is widely used by large organizations like Intel and Microsoft due to its comprehensive nature and machine-readable consistency across industries (Linux, 2023). On the other hand, CycloneDX ~~format~~standard, from the OWASP community, is known for its lightweight nature and flexibility, making it suitable for application security contexts and supply chain component analysis (OWASP, 2021). Similarly, SWID (Software Identification tags) is another ~~format~~ standard that provides a standardized way to identify and track software installations, facilitating software asset management and cybersecurity efforts (ISO/IEC, 2022). In this study, we focus on the two most prevalent and competing SBOM ~~formats~~standards: SPDX and CycloneDX (Das, 2023).

2.4 Related Work

Martin (2020), based on their industrial experience, discuss 11 crucial elements that an SBOM should have and also highlight nine possible ~~usecase~~ use case scenarios of an SBOM. In our work, we identify which ~~usecase~~ use case scenarios the current available SBOM tool ecosystems, which are built around an SBOM ~~format~~standard, address. However, instead of using Martin *et al.*'s taxonomy, we used the NTIA's official tool taxonomy, as this is a collective effort of multiple SBOM ~~formats~~standards' practitioners and SBOM tool vendors.

Xia et al. (2023b) addressed the limitation of secure SBOM sharing among software vendors in their work. They identified the SBOM sharing process of the current SBOM tools that support CycloneDX ~~format~~ standard and proposed a blockchain-based solution with verifiable credentials for secure and flexible disclosure. Their approach enhances the security, usability, and privacy of the SBOM sharing process between the software vendors.

Xia et al. (2023a) explored the importance of SBOMs in strengthening the security of the software supply chain. Through interviews with 17 practitioners and surveys of 65 respondents from 15 different countries, they identified the current challenges SBOM practitioners face. In addition to general problems related to SBOM, they found limited adoption of SBOM in the software community due to the community's lack of understanding of the technical aspects of SBOMs and the ~~usecases~~ use cases that SBOM tools offer. They highlighted that current SBOM tools suffer from issues in reliability, user-friendliness, and compliance with standard ~~formats~~standards. Furthermore, they emphasized the critical need for a robust SBOM tools ecosystem. In our work, we address these challenges by identifying the ~~usecases~~ use cases supported by the SBOM tool ecosystem, shedding light on the strengths and weaknesses of these tools.

Bi et al. (2023) perform a comprehensive survey on 4,786 GitHub discussions of projects that use SBOMs.Their work highlights key topics, challenges, and solutions intrinsic to the effective utilization of SBOMs. However, in this paper, we focus on the topics and challenges of SBOM tools instead of challenges related to SBOM usage in general.

Nocera et al. (2023)'s work is closely related to our work's RQ4, in which they examine the usage of SBOM tools in 186 Github projects. However, unlike us, they focus on studying the actual SBOM files available in these projects and how frequently they are updated, rather than analyzing the traits of the projects that employ SBOM tools. Additionally, they do not differentiate between the two ~~formats~~standards, SPDX and CycloneDX, when deriving their results, and their findings are demographic in nature.

Mirakhorli et al. (2024)'s work is closely related to our work's RQ1, in which they evaluate 84 open-source and proprietary SBOM tools. To identify their emerging use cases, they execute 5 SBOM tools and demonstrate their interoperability issues. Our study, on the other hand, identifies the ~~usecases~~ use cases of 187 SBOM tools using the NTIA's official taxonomy. Furthermore, instead of carrying out a qualitative investigation of 5 tools, we quantitatively analyze the issues of 641 SBOM tools to highlight the most prevalent problems within them. Lastly, our study extends beyond the investigation of use cases and problems of SBOM tools, as it further looks into the health factors of the projects that use these tools.

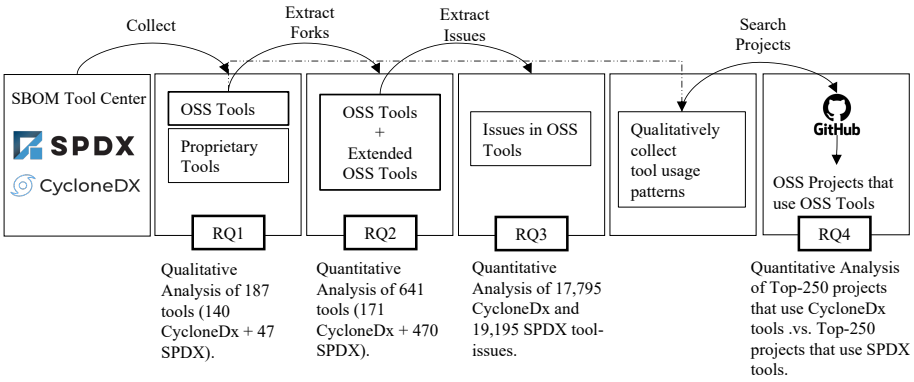


Fig. 1: Overview of our methodology and dataset.

The CHAOSS metrics that we use in RQ3 and RQ4, in order to calculate the overall health of open-source software, have been previously used to study the process and impact of onboarding new contributors in open-source software ecosystems, using an observational study and empirical validation to analyze teaching strategies, challenges, benefits, and the effects on diversity, productivity, and code quality (Foundjem et al., 2021).

3 Overview of the Study

~~We have provided~~ Figure 1 provides a general overview of our methodology ~~in Figure 1~~to address the RQs listed in the introduction. First, we collect all the SBOM tools [redacted] that are available at the tool center ~~of both formats~~ website of both the standards (SPDX [1] and CycloneDX [2]), we then qualitatively analyze these tools to identify which ~~usecases~~ use cases they support and how they are different from one another (RQ1). Second, we extend the initially collected open-source tools dataset with ~~the~~ open-source tools ~~that are~~ available on Github to compare ~~both of the formats~~ the tools' project health of both [redacted] standards (SPDX and CycloneDX) quantitatively (RQ2). Third, we extract the Github issues of all the collected open-source tools and quantitatively analyze the most prevalent type of issues (RQ3). Finally, we qualitatively curate a list of search-patterns from the previously collected list of open-source tools to identify projects on Github that use these tools~~, after~~. After collection, we study the characteristics of these projects to conclude which type of projects prefer ~~which one out~~ adopting tools of which the two ~~formats.~~ standards. Note that we normalize all metrics in our study by the repository age (in days) to account for variations in project maturity and ensure fair comparisons across repositories of different lifespans.

[1] https://spdx.dev/tools-community/
[2] https://cyclonedx.org/tool-center/

3.1 Collecting the SBOM tools (RQ1)

To investigate the different use cases of CycloneDX and SPDX tools, on 17th of January 2024, we extracted all the tools available on the official tool center websites of CycloneDX (OWASP, 2023b) and SPDX (SPDX, 2023a,b).

We found 219 tools on the CycloneDX tool center website and 47 tools on the SPDX tool center website. On closer inspection, we found some discrepancies in the information provided on the websites of both the tool centers, i.e., we found that not all the tools mentioned as open-source by their developers are necessarily open-source~~, such~~. Some of the tools have most of their functionalities private and/or commercial. Secondly, we discovered that ~~some~~ 78 of the tools in the CycloneDX [redacted] ~~also support the format, and similarly, certain tools~~ [redacted] the SPDX tool center ~~support the format as well~~are dual-format, i.e., they support both standards. We also identified certain SPDX tools whose mentioned use cases on the SPDX tool center do not necessarily provide the complete list of use cases that it supports. We also identified certain SPDX tools whose use cases, as listed on the SPDX tool center, may not reflect the complete range of use cases the tool supports. To determine the complete range of the ~~usecases~~ use cases of these tools, and to determine which ones are truly open-source, we manually analyze them.

3.2 Manual Analysis to identify ~~the usecases~~ tool use cases (RQ1)

With a 95% confidence interval and an error margin of 5%, we randomly selected 140 tools from the 219 tools that were available on CycloneDX tool center, and selected all 47 tools that were available on the SPDX tool center. This step leaves us with 187 tools to analyze in total. Two of the authors [redacted] manually investigate the tools to categorize them as open-source/proprietary and if they support a single ~~format~~ standard or both (SPDX and CycloneDX). In addition, to highlight the use cases that a tool supports, the authors categorized these tools (i.e., performed closed-card sorting (Zimmermann, 2016)) according to the official NTIA Tool Classification Taxonomy (Telecommunications and , NTIA). This tool classification taxonomy, which we discuss below, was created by NTIA after surveying several owners and SBOM tool developers to provide a variety of SBOM management use cases related to SBOM (NTIA, 2021a).

- The *Produce* category includes tools that automatically create SBOMs during software artifact creation, analyze source or binary files to generate SBOMs, and facilitate manual data entry.
  - **Build** - Tools, such as Valaa Stack[3], that automatically generate an SBOM as part of the software build process, containing information about the build.
  - **Analyze** - Tools, such as SecObserve[4], that analyze source or binary files to generate an SBOM by inspecting artifacts and associated sources.
  - **Edit** - Tools, such as CycloneDX PHP Library[5], that assist in manually entering or editing SBOM data.

[3] `https://github.com/valaatech/kernel/blob/8e24e9e731bb67d0da82fbd56b6a8e32f22c178f/CHANGELOG.md?plain=1#L841`

[4] `https://github.com/MaibornWolff/SecObserve/blob/27db653939e6209f8b6a24d3a45a76c08f358086/docs/usage/supported_scanners.md`

[5] `https://github.com/CycloneDX/cyclonedx-php-library`

- The *Consume* category includes tools that allow users to view SBOM contents in a human-readable form, compare multiple SBOMs to identify differences, and import SBOMs for further processing and analysis.
  - **View** - Tools, such as Tidelift[6], that display SBOM contents in a human-readable form, supporting decision-making and business processes.
  - **Diff** - Tools, such as SBOMTrend[7], that compare multiple SBOMs and highlight differences, such as between different versions of ~~software~~a software product.
  - **Import** - Tools, such as NetRise Turbine[8], that enable the discovery, retrieval, and import of SBOMs into systems for further processing and analysis.
- The *Transform* category includes tools that translate SBOMs between file types, merge multiple SBOM sources for analysis and audit, and support integration with other tools through APIs and object models.
  - **Translate** - Tools, such as dbom[9], that convert an SBOM from one file type to another while maintaining the same information.
  - **Merge** - Tools, such as SBOM Assembler[10], that combine multiple SBOM sources and other data for comprehensive analysis and auditing.
  - **Tool Support** - Tools, such as Enso[11], that facilitate SBOM use in other applications through APIs, object models, libraries, and other reference sources.

To avoid bias regarding the manual classification, as a first step, two authors randomly chose and categorized 29 out of 187 tools. During this round, the authors noted the different sources they had to look at to categorize the tools. The inter-rater reliability agreement for the tool categorization between the two authors yielded a Cohen's Kappa of 0.72, ~~this value~~ which indicates "substantial agreement" between the raters Gisev et al. (2013). After the first round, the authors discussed ~~each one of their approaches~~ their individual approach to categorize the tools. The authors resolved conflicts and agreed upon a ~~streamlined process to categorize the rest of the tools. The authors agreed upon a~~ unified search approach to continue their manual analysis on the rest of the tools.

After agreeing upon the categorization criteria and results of the 29 tools, our manual analysis for the rest of the 158 tools entails reading the tool description available on the CycloneDX or SPDX center website and some additional exploration based on the types of tools: open-source or proprietary. In regards to open-source tools, we read through the Readme of their GitHub repository. In case the Readme did not provide sufficient information, we searched the tools' GitHub repository with the following keywords separately: "sbom", "spdx", and "cycloneDx", [redacted] then read the relevant Markdown or Text ~~format~~ standard files if available. If there is no such file available, then we look at the usage of "spdx" and

[6] https://support.tidelift.com/hc/en-us/articles/4406286154004-About-projects-and-bill-of-materials#obtaining-a-bill-of-materials-0-1

[7] https://github.com/anthonyharrison/sbomtrend

[8] https://www.netrise.io/platform/software-bill-materials-management

[9] https://github.com/HaRo87/mdbom

[10] https://github.com/interlynk-io/sbomasm

[11] https://knowledge.enso.security/connectors-and-integrations/connectors-and-marketplace

"cycloneDx" keywords in the code of these repositories. The code usually provides us ~~with an idea of a tool if it~~ a clear idea about whether a tool is capable of generating, consuming, or transforming an SBOM. The code also highlights if a tool can support both ~~formats~~ standards or just one.

In regards to proprietary tools, we search for the keywords "sbom", "spdx", and "cycloneDx" initially on the landing web-page of the tool~~. This~~, which would sometimes provide us with information on the use cases and ~~format~~ standard support ~~type~~ of the tool. If ~~we are notable to find the relevant information~~not, we search the same keywords along with the tool name on the Google search engine. This search would yield ~~our~~ information from a variety of sources ~~spanning~~ from LinkedIn or Medium blogposts written by the tool developers to the tool's documentation and demo tutorials. In case we find a tool demo or documentation, we read through it to understand its use cases, while in the other case, when there is only a blog post available, we read through the blog post to understand what the tool does.

Both ~~formats~~standards' tool-centers support tags along with tool names as well~~, while~~. While these helped us identify the primary ~~usecase~~ use case of the tool, on ~~a~~ closer inspection, we found that the tags do not ~~sufficiently provide the~~ provide a sufficiently complete picture of which ~~usecases~~ use cases the tool supports. Both of the tool centers also had some duplicate tools ~~but they were difficult to identify due to~~ with different naming conventions~~, therefore,~~. Therefore, we mapped the official urls of the tools and identified 15 duplicates, with 1 exception in which one url carried two different tools.

In this final round of manual analysis, the inter-rater reliability agreement yielded a Cohen's Kappa of 0.85, ~~this value~~ which indicates "nearly perfect agreement" ~~Gisev et al. (2013). To finalize~~ (Gisev et al., 2013). To remaining conflicts in the classification of ~~the tools , the authors discussed and resolved their conflicts~~tools were resolved through negotiated agreement (Campbell et al., 2013). In Table 10, we present the results of 170 distinct tools, out of which 62 are proprietary and 108 are open-source~~tools~~. This table lists open-source tools first and proprietary tools later, all tools sorted by their names in alphabetical order.

3.3 Extending the collection of OSS SBOM tools (RQ2)

To collect open-source SBOM tools that support SPDX and CycloneDX, we ~~extracted the forks of the~~ expanded on the collection of current open-source ~~tool Github repositories that were~~ tools available on SPDX and CycloneDX tool-center by extracting their forks from the Github repositories. One of the authors manually analyzed the forked repositories to identify contributing-forks and filtered out forks that did not expand over the parent tool i.e., have no further commits after the fork (non-contributing forks), and also filtered out the forks that did not [redacted] a description in the Readme file from which we can understand if it is a tool repository or not.

As a result, collectively from the tool-center and the contributing-forks, we found 171 CycloneDX repositories and 470 SPDX repositories that represent a tool project. ~~Even though, when compared to~~ From the tool-center, we found [redacted] open-source tools ~~, our Github search reveals more~~ than SPDX ~~tools (~~ [redacted] after collecting

the contributing-forks, we have 470 ~~) than~~ open-source SPDX tools ~~(~~compared to 171 CycloneDXtools. A larger ecosystem of tools could suggest a wider variety of functionalities and potential benefits for users in managing software bill of materials (SBOMs) effectively.

3.4 Extracting the issues and their tags of the OSS SBOM tools (RQ3)

We queried the 171 CycloneDX and 470 SPDX tool Github repositories to collect their issues. Since not all the repositories contain issues, we identified 17,795 issues in 90 CycloneDX tool repositories and 19,195 issues in 191 SPDX tool repositories. Since CycloneDX's first prototype was released in May 2017, we filter out all the issues that were created before 2018. After this filtration step, we were left with 17,093 closed and open issues in 90 CycloneDX tools and 16,747 closed and open issues in 171 SPDX tools.

For analyzing the types of issues, we consider only those issues that are already tagged by tool maintainers on Github. This step leaves us with 19,812 [redacted] tags associated to 16,747 issues of 171 SPDX tools (with 361 unique tags), and 22,302 [redacted] tags in 17,039 issues associated to 90 CycloneDX tools (with 360 unique tags).

We sort the tags by their number of occurrence and investigate their distribution. To investigate the most prevalent tags only, we consider the fourth quartile of this distribution. For CycloneDX, we find that the 92 ~~unique tags that~~ most common tags are associated with 27 to 5815 issues. For SPDX, we found that the 91 ~~unique tags that~~ most common tags are associated to 28 to 3979 issues. The first author then manually investigated these 92 CycloneDX and 91 SPDX tags to exclude the ones that do not convey anything regarding the meaning of an issue ~~by~~ in their name, such as "P1", "minor", "cannot reproduce", "super", "easy", and "not an issue", etc. ~~This filtration leaves us with 49 most prevalent unique tags in tools and 48 most prevalent unique tags in tools.~~ Furthermore, tags with the same meaning were merged, such as "technical debt" and "technical-debt", "license scan" and "license", and "new feature" and "nice to have" etc. This process leaves us with the 49 most prevalent unique tags in CycloneDXtools and 48 most prevalent unique tags in SPDXtools. The complete list of tags and their categories is available on our replication package[12].

After merging the tags, we manually ~~group the similar tags in a broader category~~abstract similar tags into broader categories. For this process, the first author sorts the tags and groups the ones that have similar purposes, such as "defect", "technical issue" and "bug", categorized as "Bug fixes and Defects", docs and documentation categorized as "Documentation", and "enhancement", "must have", "nice to have" and "feature", categorized as "Feature Development and Enhancement". The last author then, having the most experience, re-analyzed the categories of the tags and raised 10 concerns about the grouping scheme. These concerns were related to: the possible duplication of tags, such as CycloneDX having ~~two~~ duplicate tags named "stale"; mis-judgement of categories e.g., "docs" falling into "CI/CD" category instead of "Documentation"; and better naming

[12] https://github.com/SAILResearch/wip-24-ali-sbom-emse

convention, e.g., “Bug Fix” should be “Bug Fixes and Defects”. Through a collaborative process, the authors refined ~~and finalized 14 distinct categories con[illegible]ng of 97 tags (CycloneDX~~the 97 tags (CycloneDX : 49, ~~SPDX~~SPDX : 48) into 14 distinct categories, as detailed in Section 4.3.

3.5 Collecting OSS projects that use OSS SBOM tools (RQ4)

~~In order to~~ To collect projects that use SBOM tools, ~~we~~ could have selected Github projects with an SBOM file in their repository. H[illegible]ver, projects typically do not release their SBOM files in ~~the repository because these~~ [illegible]ir version control repositories. Instead, SBOM files are generated as an outcome of the build process within ~~their build system~~[redacted] and reside in a GitHub release or dependency managem[illegible] repositories, such as Ma[illegible] Central~~carries SBOM [illegible] for Java projects~~. In both cases, one cannot easily query projects to find only those containing an SBOM file. S[illegible] we wanted to determine which SBOM tools are being used ~~to generate SBOMs, we~~ [illegible] the Github projects in general, we instead searched for traces of SBOM ~~tool initialization in the repositories of GitHub projects~~tools’ initialization code in the code files of GitHub project repositories.

~~Out of the~~ To identify the initialization code of each SBOM tool, we analyze the code-base and usage guidelines in the Readme files of all the open-source SBOM tools that are available on the official tool-center website of SPDX and CycloneDX ~~,~~ (analyzed in RQ1) and also support the SBOM creation (Build) ~~usecase~~ use case from the NTIA [illegible]ol taxonomy~~, we analyze their codebase and the usage guidelines in their Readme file to identify the source-code command they require to initiate an SBOM file in a project. We call such commands *creation-patterns*. We identified~~. We identified the initialization code of 66 ~~creation-patterns for these~~ open-source SBOM tools (16 for SPDX tools and [illegible]or CycloneDX tools). All of ~~the creation-patterns~~ t[illegible] initialization codes are available in our replication package, however, a few of them are displa[illegible] ~~along-with-their~~, along with their tool nam[illegible] Table 9. The initialization codes may include dependency code of an SBOM tool, path to an SBOM tool [illegible]ary or a web-service, or installation [illegible]mand of an SBOM tool. To identify projects that use these SBOM tools~~in order to create SBOM files~~, we then use the Github search API to identify projects that carry ~~these creation-patterns~~ the initialization code of a tool in their code files or [illegible]d configuration files.

~~and Tools and corresponding pattern used to find repositories using these tools.~~

~~**Tool Name Code Pattern for SBOM creation**bom ort protobom sbom-tool spdx-sbomgenerator tools-golang cyclonedx-cli cyclonedx-conan cyclonedx-core-java cyclonedx-dotnet~~

~~cyclonedx-gomod cyclonedx-go cyclonedx-node-module cyclonedx-node-npm cyclonedx-node-pnpm cyclonedx-php-composer cyclonedx-php-library cyclonedx-python-lib cyclonedx-ruby-gem cyclonedx-rust-cargo cyclonedx-webpack-plugin~~

The search queries for the projects that use SPDX tools or CycloneDX tools to create SBOMs returned 3,944 repositories. After removing duplicates and the source repositories of the tools themselves, we ended up with 2,426 repositories in total, comprising 307 projects that use SPDX tools, 1087 projects that use CycloneDX tools, and the remaining projects that use both SPDX and CycloneDX tools. In our analysis, to compare SPDX and CycloneDX, we consider projects that

Table 1: Manual classification of 170 SBOM tools (19 exclusively support SPDX, [illegible]xclusively support CycloneDX, while 78 support both ~~formats~~standards) in NTIA's tool taxonomy.

Category	Type	Description	SPDX	CycloneDX
Produce	Build	SBOM creation	74.36%	79.35%
	Analyze	Artifact analysis	70.51%	57.61%
	Edit	Manual SBOM editing	8.97%	6.52%
		Total	94.87%	92.39%
Consume	View	Visualization of decision making	67.95%	46.74%
	Diff	SBOM comparison	10.26%	6.52%
	Import	SBOM retrieval	43.59%	23.91%
		Total	79.49%	55.43%
Transform	Translate	File type conversion	14.10%	15.22%
	Merge	SBOM aggregation	11.54%	7.61%
	Tool Support	Integration support	37.18%	52.17%
		Total	46.15%	63.04%

use only one of the two ~~formats~~ standards (1, 394). Of these 1, 394 projects, the first set of 307 projects uses SPDX tools, while the second set of 1087 projects uses CycloneDX tools. We then sort the projects by their number of [illegible]Hub stars and select the Top-250 projects from each set. Selecting projects based on the number of GitHub stars ensures a focus on popular and well-supported projects, reflecting community interest and active development (Borges and Valente, 2018).

~~To summarize~~Finally, we compare ~~two sets, the first set representing~~ the project health of the [redacted] Top-250 ~~projects~~ (by stars) projects that use CycloneD[illegible]ols ~~, and the second set representing with~~ Top-250 ~~projects~~ (by stars) projects that use SPDX tools.

4 Results

4.1 ~~Usecases~~ use cases of the SBOM ~~Tools Ecosystem~~ Tool Ecosystems (RQ1)

In RQ1, we study the ~~usecases~~ use cases of the open-source SBOM ~~tools that support the~~ tool ecosystems of CycloneDX and SPDX ~~formats. Identifying different use cases~~ standards. Investi[illegible]ng which use cases each tool ecosystem support and what are the competing use cases in each tool ecosystem would help us [illegible]lerstand their different focus. A clear understanding of each tool-ecosystem's strength and [illegible]knesses in terms of the use cases that it supports would allow the SBOM practitioners to compare the ~~tools and adopt or modify~~ standards and adopt the most suitable ones for their needs. Such an identification process ~~will~~ would also highlight the capabilities and limitations of the available SBOM tools, which ~~will~~ would help SBOM tool makers improve their tools.

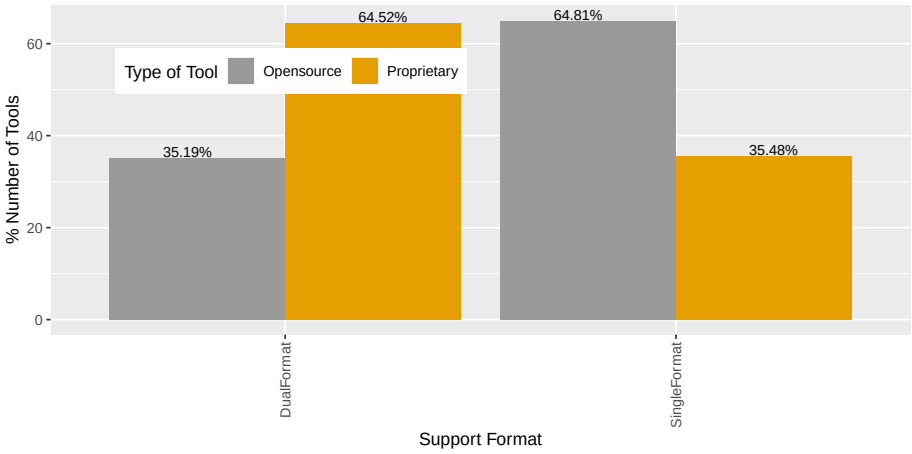


Fig. 2: Distribution of 170 manually analyzed SBOM tools across the SBOM specifications that they support. DualFormat tools support both standards, while SingleFormat tools support a single standard.

4.1.1 SPDX ~~-~~vs. CycloneDX Tools.

Proprietary tools have a higher tendency to support ~~for dual-format~~ multiple SBOM standards [redacted] than open-source tools. Across the 170 tools that we manually analyzed, 78 tools support both SPDX and CycloneDX , even if they are mentioned on the tool-center webpage of only one of the ~~formats. Eventually, for~~ standards. For comparison, we group the tools into ~~three categories:~~ two categories: single-format tools that support either SPDX ~~only (19), or~~ CycloneDX ~~tools that support only (73) ,~~ [redacted] (92) and dual-format tools that support both ~~formats~~ standards (78). [redacted]

~~This figure shows that~~ Figure 2 shows that among the tools that explicitly support either CycloneDX ~~and~~ or SPDX ~~mostly adopt the~~ standard, open-source ~~format, while dual-format tools are mostly proprietary~~ is the most prevalent choice. While among the tools that support dual-format, proprietary is the most prevalent choice. In other words, proprietary tools are more inclined towards supporting both ~~formats~~ standards (64.52%) while open-source tools ~~are~~ more inclined towards supporting a single ~~format~~ standard (64.81%).

RQ1: Open-source vs. Proprietary in terms of standard support.

Proprietary tools are more inclined towards supporting both standards (64.52%), while open-source community tools are more inclined towards supporting a single standard at a time (64.81%). [redacted]

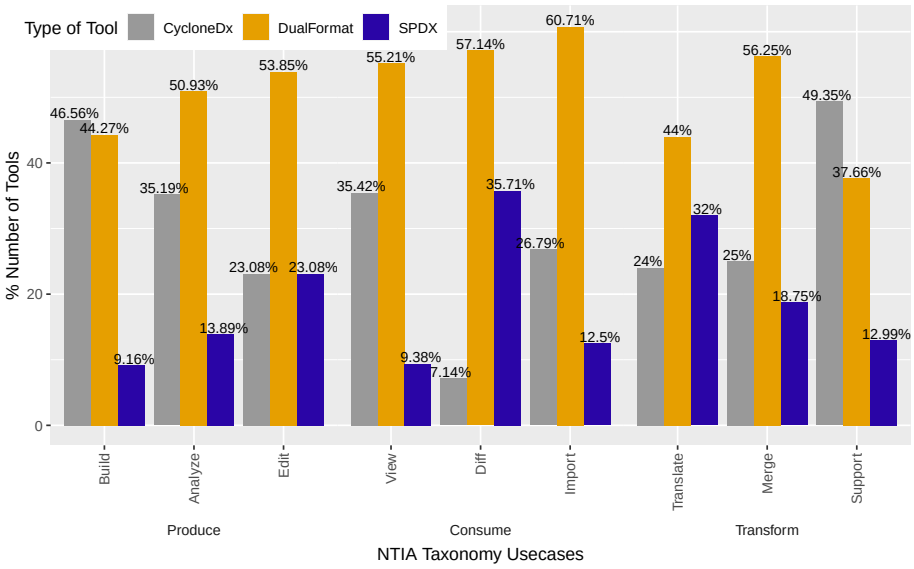


Fig. 3: [redacted] distribution of 170 manually analyzed to[redacted] across Open-source ~~.~~vs. Proprietary. DualFormat tools support both standards.

CycloneDX tools lack support for the *Diff* ~~usecase~~use case, dual-format tools lack support for the *Support* ~~usecase~~use case, and SPDX tools lack support for the *Build* ~~usecase~~use case. Furthermore, comparatively, CycloneDX tools have a greater support for the *Build* and *Support* ~~usecase~~use case, dual-format tools have a greater support for the *Import* ~~usecase~~use case, while SPDX tools have a greater support for the *Diff* and *Translate* ~~usecase~~use case. In Table 1, we show which use-case from the NTIA tool taxonomy is prevalent the most in the tools of which standard. We now look into the ~~usecases~~ use cases that each of the ~~tool-category~~ categories (CycloneDX, SPDX, dual-format) support. The results of this analysis are represented in Figure 3. Although the figure shows that SPDX tools are behind in every ~~usecase~~use case, the interpretation of SPDX lagging behind the other ~~formats~~ standards might not be entirely correct especially provided the fact that there are only 19 tools that explicitly support SPDX, as compared to 73 for CycloneDX and 78 for dual-format support. For a fair comparison between the distributions, we consort towards the chi-square test of random distribution. The chi-square test found a significant difference between the tools~~by their support~~ ~~that across the usecases that they support~~' support standard and the tools' use cases, with < 0.05 significance level, and a "Strong" Cramer's effect size of 0.181 (Kampenes et al., 2007).

Since the tools ~~by their support type are significantly different~~ support standard significantly differ in their coverage of ~~usecases~~use cases, we further investigate the Cramer's effect size to identify the ~~usecases~~ use cases that have a strong to weak or negative to a positive association with a certain support ~~format~~standard. To

Table 2: The Pearson residuals and contribution proportion (%) comparison between the use cases of SPDX, CycloneDX, and dual-format (DF) SBOM tools.

Cateogry	use case	SPDX	CycloneDX	DF	SPDX (%)	CycloneDX (%)	DF (%)
Produce	Build	-1.334	1.733	-0.812	5.054	8.530	1.872
	Analyze	0.129	-0.362	0.248	0.047	0.372	0.174
	Edit	0.949	-0.84	0.236	2.555	2.005	0.158
Consume	View	-1.085	-0.304	0.831	3.341	0.263	1.962
	Diff	2.275	-1.848	0.421	14.688	9.695	0.502
	Import	-0.19	-1.29	1.222	0.103	4.722	4.239
Transform	Translate	2.533	-1.09	-0.374	18.213	3.371	0.398
	Merge	0.58	-0.806	0.399	0.956	1.846	0.451
	Support	-0.107	1.729	-1.449	0.032	8.488	5.963

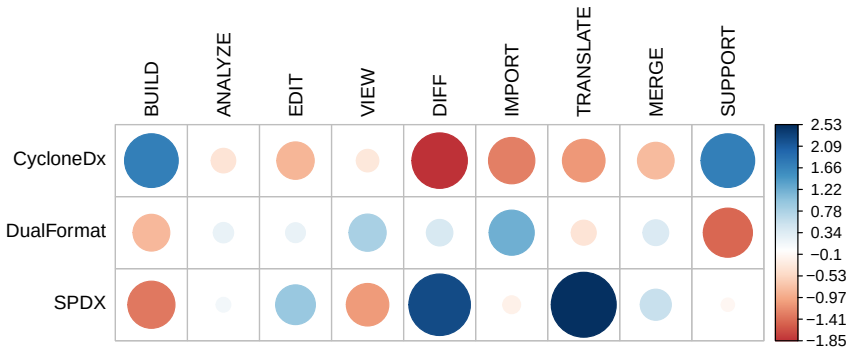


Fig. 4: Visualization of the Pearson residuals to compare the use cases of SPDX, CycloneDX, and dual-format SBOM tools. A positive residual value indicates a positive association, while a negative residual value indicates a negative association between a support standard and a use cases. The size of the circle indicates strength of the association of a support standard with a use case.

investigate the associations, we calculate Pearson residuals of each observation from the chi-square test. In this case, the Pearson residual highlights the strength of the association between a ~~usecase to the tool type by its support format~~use case and a support standard. We present the residual values in Table 2, and visual representat of the same data, for an easier interpretation, in Figure 4.

The figure shows that CycloneDX tools are negatively associated ~~("not associated")~~ with the *Diff* ~~usecase~~use case. Dual-format tools are negatively associated with the *Support* ~~usecase~~use case. Furthermore, SPDX tools have a lack of support for the *Build* ~~usecase~~use case. On the other hand, CycloneDX tools are positively associated with the *Build* and *Support* ~~usecases~~use cases, dual-format tools are positively associated with the *Import* ~~usecase, and there is an attraction between~~ use case, and SPDX tools ~~and~~ are positively associated to the *Diff* and *Translate* ~~usecase. The size of the circle in the figure represents the contribution of a tool type towards a usecase.~~ use case.

We now use the resi values to investigate which ~~usecases~~ use cases have contributed the most towards the chi-square value (the difference between the tools

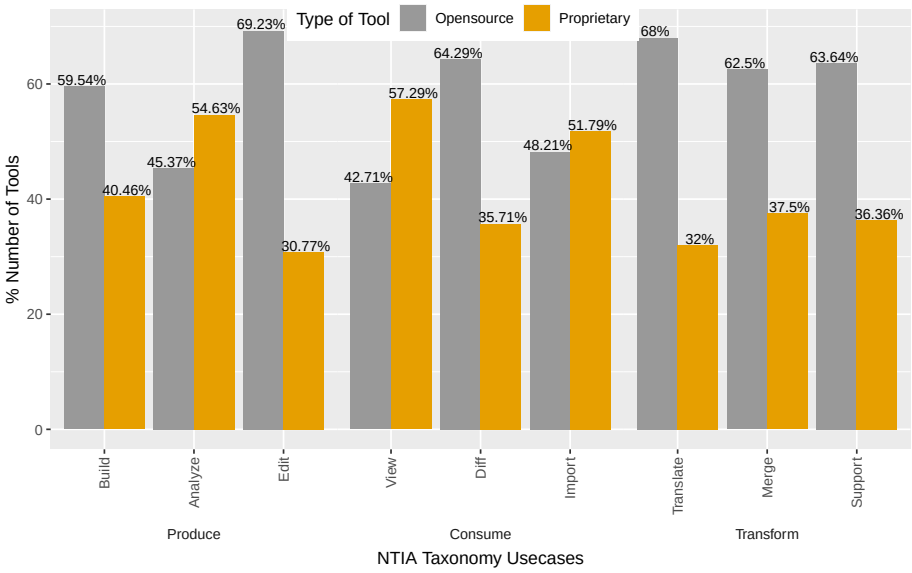


Fig. 5: Comparison of how 170 SBOM tools (both open-source and proprietary) distributed across the NTIA's use-case taxonomy.

based on the ~~format~~ standard they support)~~, this result will~~ . This result would provide us with an insight into the ~~usecases whose frequency is mainly different~~ use cases whose frequency differs substantially between the SPDX, CycloneDX, and dual-format tools. The residual values contribution (%) is presented in Table 2. From this table, it can be observed that the most contributing tool-type/use-case residuals to the chi-square are *Translate* (21.58%), *Diff* (24.38%), and *Build* (13.58%). These three ~~usecases~~ use cases contribute 59.54% to the total chi-square value and thus account for most of the difference between SPDX, CycloneDX, and dual-format tools.

RQ1: Supported use cases of SPDX vs. CycloneDX.

We find that tools that explicitly support SPDX mostly support the *Translate* use case and lack support for the *Build* use case. We also find that tools that explicitly support CycloneDX mostly support the *Build* use case, while lack support for the *Diff* use case. Finally, tools that support dual-format mostly support *Import* use case, while lack support for the transofmration-*Support* use case. These results help SBOM tool-developers understand the distinct strengths and weaknesses of the tools' ecosystem of SPDX and CycloneDX, guiding them in tailoring their tools to better cater to specific use cases.

4.1.2 Open-source ~~-~~vs. Proprietary SBOM Tools.

Proprietary SBOM tools across SPDXand CycloneDX offer more use cases than the open-source SBOM tools. We manually identify the NTIA ~~'s taxonomy usecases that~~ taxonomy use cases supported by 108 open-source ~~tools~~ and 62 proprietary ~~tools support and compare them to find out if the~~ SBOM tools, comparing their coverage to assess whether open-source tools are as ~~resource~~ ~~as the~~ comprehensive as proprietary ones. ~~Initially, we ran the~~ A Chi-square test of independence ~~and found~~ reveals a statistically significant difference between the ~~open-source tools and the proprietary tools, with~~ two, with a p-value of < 0.05 ~~significance level,~~ and a "Strong" Cramer's effect size of 0.172 (Kampenes et al., 2007).

Proprietary tools moderately support the *Analyze* and *View* ~~usecases~~use cases, while ~~the~~ open-source tools lack support for these~~usecases~~. ~~On the other hand~~Conversely, open-source tools ~~have a greater availability for~~ excel in the *Support* ~~usecase~~use case, ~~while the~~ where proprietary tools ~~have a lack of~~ show limited availability~~for this usecase~~. ~~Since the proprietary and open-source tools are significantly different in their coverageof use-cases~~Given the significant difference in use case coverage, we further investigate the Cramer's effect size to ~~investigate which type of tools have a strong to weak, or negative to positive association with a certain usecase. To investigate the associations~~explore the strength of association between tool types and specific use cases. As previously explained, we calculate Pearson residuals ~~of each observation from the chi-square test. The Pearson residual highlights the most contributing factors towards the final score of a chi-square test , which means in our case, that it would highlight~~ to identify which factors most contribute to the Chi-square test results, reflecting the strength of ~~association between a usecase to proprietary and open-source tools. We present the residual values~~ the association between use cases and tool types.

The residual values are presented in Table 3 ~~, and for an easier interpretation, a visual representation of the same data~~ and visually represented in Figure 6. In ~~this figure, the negative residuals are in redindicating~~ the figure, negative residuals (in red) indicate a lack of support (negative association)~~between the corresponding usecase and tool type (open-source or proprietary) . The figure shows~~ , while positive residuals (in blue) indicate support (positive association). The results show that open-source tools ~~are negatively associated (lack support)~~ lack support for the *Analyze* and *View* ~~category. On the other hand, there is a lack of support between proprietary tools and the usecases~~ use cases, while proprietary tools are weakly associated with the *Build*, *Support*~~, and *Translate*. Similarly, the positive residuals are in blue indicating support (positive association) between the corresponding usecase and tool type (open-source or proprietary). The figure also shows that open-source tools are positively associated to *Build*, *Support*,~~ and *Translate* ~~. Furthermore, proprietary~~ use cases. Proprietary tools moderately support *Analyze* and *View*~~usecases.~~ .

The size of the ~~circle accounts for the contribution of a tool type towards a usecase. Since some of the usecases have a positive association with the type of tool (open-source or proprietary), we now investigate the contribution of each type towards a particular usecase in terms of percentage. The residual valuesproportion for each usecase is~~ circles in the figure reflects the contribution of each tool type to

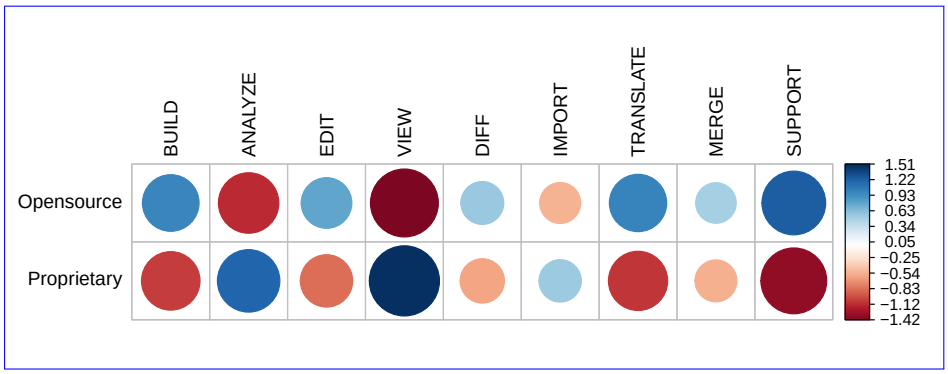


Fig. 6: Visualization of the Pearson residuals to compare the use cases of Open-source vs. Proprietary SBOM tools.

Table 3: The Pearson residuals and contribution proportion (%) comparison between the use cases of Open-source .vs. Proprietary SBOM tools.

Cateogry	use case	Proprietary	Open-source	Proprietary (%)	Open-source (%)
Produce	Build	-1.052	0.986	6.027	5.29
	Analyze	1.199	-1.123	7.825	6.868
	Edit	-0.842	0.789	3.864	3.391
Consume	View	1.512	-1.416	12.444	10.922
	Diff	-0.604	0.565	1.983	1.741
	Import	0.552	-0.517	1.659	1.456
Transform	Translate	-1.078	1.01	6.329	5.555
	Merge	-0.541	0.507	1.592	1.397
	Support	-1.332	1.248	9.662	8.48

a given use case. To quantify these contributions, we calculate the residual values' proportions for each use case, presented in Table 3. ~~From the proportions, it can be observed that the most contributing tool-type/use-case residuals to the chi-square results~~ The top contributors to the Chi-square result are *View* (~~23.36%~~23.36%), *Analyze* (~~14.69%~~14.69%), and *Support* (~~18.41%). These three usecases contribute 56.46% to the total chi-square result and thus account for most of the~~ 18.41%), which together account for 56.46% of the total Chi-square difference between open-source and proprietary tools.

Table 4: Mann-Whitney U test (Wilcoxon et al., 1970) applied to evaluate the difference between normalized community-health metrics of the 171 CycloneDX tool-repositories and 470 SPDX tool-repositories. $p - value < 0.05$ indicates a significant difference between the two distributions.

Metric	p-value	Cliff's Delta	Effect Size (Wan et al., 2019)
ClosedPullRequest	1.00^{-10}	0.595	Large
Commits	7.28^{-07}	0.649	Large
Contributors	2.65^{-15}	0.726	Large
Forks	2.96^{-10}	0.619	Large
OpenPullRequest	8.48^{-10}	0.476	Large
Releases	5.47^{-09}	0.448	Medium
Stars	3.80^{-12}	0.643	Large
Watchers	1.50^{-16}	0.737	Large

RQ1: Open-source vs. Proprietary contribution towards use cases.

In general, despite being fewer in number compared to open-source tools, proprietary SBOM tools provide 1.14 times more use cases. Our analysis reveals that the use case *View* has a strong positive association with proprietary tools, while *Support* is more strongly associated with open-source tools. Conversely, proprietary tools show a lack of support for *Support*, and open-source tools struggle with *View*. Our results suggest that the OSS community should prioritize developing tools that visualize SBOMs for decision-making, while proprietary tools could focus on creating more libraries and APIs to enhance integration support.

4.2 State of the OSS SBOM tools (RQ2)

In RQ2, we inspect the health of the open-source SBOM ~~tools, we analyze~~ tool ecosystem by analyzing different CHAOSS health measures[13], such as the number of contributors, issues, forks, stars, watchers, pull-requests, releases, and the rate of returning contributors. This analysis ~~will~~ would reveal the level of community engagement, activity, and sustainability of open-source SPDX and CycloneDX tools.

4.2.1 Insights on the community-health factors of SBOM tool ~~repositories~~ecosystem.

All ~~the~~ open-source community health factors are significantly ~~different from each other in and~~ higher for CycloneDX than for SPDX, with a medium to large Cliff Delta effect size. To assess the overall health and activity of the identified tool repositories, we collect ~~their~~ key metrics, including the number of stars, forks, releases, contributors, watchers and the counts of open/-closed issues ~~, and open/closed~~ and pull requests. ~~These metrics have previously~~

[13] https://chaoss.community/kbtopic/all-metrics

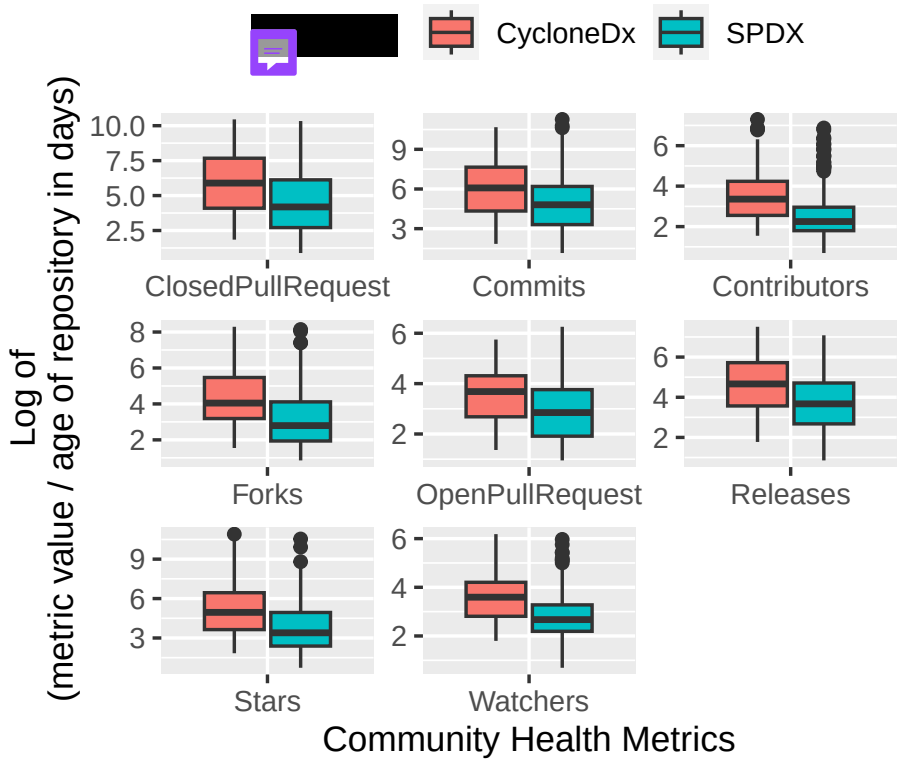


Fig. 7: Comparison of the distribution of normalized metrics (by repository age), i.e., #PRs, #Contributors, #Stars, #Watchers, #Releases, #Forks representing the community-health metrics of 470 SPDX and 171 CycloneDX tool repositories.

~~been used to assess many open-source community health factors of software repositories, such as~~ Since every repository has a different maturity level, we normalize the collected values of the metrics by the age of repository (#days). These metrics are commonly used to evaluate activeness, maturity, ~~overall maintenance activity to issues~~maintenance efforts, and user engagement ~~of the tool repositories~~in open-source communities (Zerouali et al., 2019; Foundjem et al., 2021).

Figure 7 ~~represents the result of what we extracted~~ illustrates the results from the open-source ~~Github~~ GitHub tool repositories of 470 SPDX tools and 171 CycloneDX tools. To compare the difference across the metrics between the tools of the two ~~formats, we applied~~ standards, we apply the Mann-Whitney U test (Wilcoxon et al., 1970) ~~, on the non-normal distributions of the extracted popularity metric values~~to the normalized popularity metrics, followed by ~~the~~ a Bonferroni correction to ~~reduce the chance of false positive~~mitigate the risk of false positives. The results of this test are ~~available~~ presented in Table 4, ~~which shows p-value< 0.05 to indicate~~ with p-values¡0.05 indicating a significant difference and Cliff's Delta ~~value to indicate~~ values indicating the effect size of this difference (Tomczak and Tomczak, 2014).

These results show ~~that all the~~ significant differences in all community-health metrics ~~are significantly different in both formats. Furthermore,~~ between the two standards, with the Cliff's Delta effect size ~~suggests that these differences are~~ indicating medium to large ~~.~~

~~Since all the metrics significantly differ in both formats and because we have an~~ differences. Given the unequal number of tools ~~from each format, to compare the metrics in the tools of each format~~in each standard, we performed 1,000 bootstrap samples with a ~~1000 samples bootstrapping on the tools with a 95%~~ 95% confidence interval ~~to~~ determine the true median for each metric~~in both of the formats. We normalize the true median value by the age of each repository to cater the old repositories in our dataset~~. Next, we look deeper into the different CHAOSS health measures[14] that were studied.

~~*Comparing the stars, watchers, and forks of and tools.*~~ ~~**tools have more number of stars, watchers, and forks than .**~~ ~~Borges and Valente (2018) performed a comprehensive empirical investigation, *"What's in a GitHub Star?"*,~~ Borges et al. Borges and Valente (2018) conducted an empirical study on GitHub projects~~and found that~~, identifying three main reasons why developers star a project~~mainly for three reasons: to show appreciationfor a project, bookmark a project~~ : to express appreciation, to bookmark the project for later use, or because they are ~~using the project. They further found that 3 out of 4~~ actively using it. They also find that 75% of developers consider the number of normalized stars before contributing to a ~~GitHub project. Finally, they recommend project owners compare the number of stars of their projects with competitors to assess how desirable their projectwill be~~ project. Based on this, they suggest that project owners monitor their star count relative to competitors to gauge their project's attractiveness to the open-source community. ~~In terms of Stars~~When comparing CycloneDXand SPDXtools, CycloneDX tools ~~exhibit higher median 62.3~~ have a statistically higher median star count (62.3) compared to SPDX tools ~~5.23. Based on higher number of Stars (Borges and Valente, 2018), our results suggest that~~ (5.23). While this suggests that CycloneDXtools may have garnered more attention from the open-source ~~developers may desire contributing more to tools than tools.~~ community, the difference could be influenced by factors such as the age of the repositories, the number of contributors, or the broader use of the tools in specific domains. Therefore, the higher star count does not necessarily indicate a greater likelihood of contributions but may reflect other factors such as visibility or project maturity.

Moreover, developers or users watch a repository ~~in order~~ to receive notifications about its activity, such as new issues, pull requests, or discussions. Similar to stars, ~~in terms of watchers , despite having a lesser number of~~ the number of watchers reflects the level of engagement ~~with~~ a project. Despite CycloneDXhaving fewer open-source tools than SPDX, its median number of watchers ~~36.2 significantly~~ (36.2) exceeds that of SPDX ~~14.4. Number of watchers suggest the rate of observers of projects in~~ (14.4). This suggests that CycloneDXtools attract more attention from observers in the open-source community (Borges and Valente, 2018)~~, and we find that tools have far more observers than tools.~~

[14] https://chaoss.community/kbtopic/all-metrics

~~The objective of a forkmetric is to ascertain~~ . The "fork" metric, on the other hand, measures how many copies of a project exist on a code development platform (C. P. L. Foundation, Accessed May 27, 2024). In ~~terms of forks, once again~~ , this case, CycloneDX tools again outperform SPDX, with a median number of ~~forks 27.9 surpasses~~ regular forks (27.9) significantly higher than that of SPDX ~~median number of forks 3.25~~(3.25). This indicates greater interest in CycloneDX tools, not only in terms of observation but also in terms of users creating personal copies for further development.

~~*Comparing the PRs, releases, contributors, and contributions of and tools.*~~ **CycloneDX tools have more ~~number of~~ PRs, releases, contributors, and contributions than SPDX.** Pull Requests (PRs) are a common ~~method~~ mechanism for proposing changes to source code repositories, ~~allowing~~ enabling stakeholders to review and discuss ~~proposed changes~~ modifications before integration (Chopra et al., 2021). CycloneDX surpasses SPDX in both the median ~~number of~~ and mean number of normalized open and closed pull requests~~.~~, suggesting a higher level of ongoing development and code review activity. Specifically, CycloneDX ~~number of open pull requests (median:6.84 and~~ has a median of 6.84 open PRs (mean: 28.2)~~compared to~~ , whereas SPDX ~~(median :~~has a median of 0 ~~and~~ (mean: 10.2)~~, and number of closed pull requests~~ . Similarly, the number of normalized closed PRs in CycloneDX (median: 66~~and mean: 1424) compared to~~ , mean: 1,424) significantly exceeds SPDX (median: 0~~and~~ , mean: 10.2). ~~Similarly,~~

A similar trend is observed in commit activity: CycloneDX tools ~~' number of commits (median :~~have a higher median (438~~and mean:~~) and mean (2,453) ~~also surpass~~ number of normalized commits compared to SPDX tools ~~' commits~~ (median: 122~~and~~ , mean: 1,254)~~.~~

~~Number of project releases suggest the level of development~~ , indicating more frequent updates and iterative improvements. The number of normalized project releases serves as an indicator of development progress, update frequency, ~~updates,~~ and versioning ~~of a software project. For this metric as well~~practices. Here too, CycloneDX ~~tools~~ exhibits higher values (median: 0~~and~~ , mean: 125) ~~surpass~~ than SPDX ~~tools~~ (median: 0~~and~~ , mean: 26.8), implying more active release cycles.

~~The number of contributors on GitHub~~ Finally, contributor count reflects the collaborative nature ~~of a project and the level of community engagement . Similar to the~~ and community engagement of a project. In line with other metrics, ~~in terms of contributors,~~ CycloneDX tools attract more normalized contributors (median: 28.2~~and~~ , mean: 88.8) ~~surpass~~ than SPDX tools (median: 9.25~~and~~ , mean: 34.5), suggesting broader community involvement. While these quantitative differences shed light CycloneDX's relative dynamism, future work could explore underlying qualitative factors—such as governance structures, funding, or adoption trends—that might explain why CycloneDXexhibits greater development activity and engagement than SPDX .

Table 5: Result of Mann-Whitney U Test to compare the difference between returning contributor groups for SPDX and CycloneDX tool repositories.

Groups	p-value	Cliff's delta	Effect size	Significance
1-10	0.000	0.084	negligible	True
10-100	0.423	-0.031	negligible	False
100-1k	0.387	-0.058	negligible	False
1k+	0.710	0.102	negligible	False
all	0.000	0.116	negligible	True

RQ2: Community-health of tools

The number of normalized stars, watchers, and forks suggests that, despite having fewer tools than SPDX, the open-source community is more engaged with CycloneDX tools. The higher number of stars further indicates that open-source developers may prefer contributing to CycloneDX tools over SPDX tools (Borges and Valente, 2018). Similarly, across all key metrics—Pull Requests, Issues, Contributors, and Releases—CycloneDX tools exhibit greater development activity than SPDX tools, despite their smaller toolset.

~~*Comparing the returning contributors of and tools.*~~ **There is a negligible difference between the rate of returning contributors of CycloneDX tools and SPDX tools.** A returning contributor of a project is a Github contributor who contributes to a project more than once. We group the returning contributors by their number of contributions (normalized commits) and compare them between the two ~~formats~~ standards (SPDX and CycloneDX) to investigate which ~~format~~ standard tools have a better rate of returning contributors. Table 5 presents the Mann-Whitney U test statistics for the difference of SPDX and CycloneDX tools across the returning contributor groups (by number of commits), including Bonferroni corrected p-values, Cliff's delta, effect sizes, and significance level of p-value.

We categorized repositories into four groups based on the number of normalized commits contributed by each contributor ($> 10^0$, $> 10^1$, $> 10^2$, $> 10^3$ – commits). For the small commit count group (~~1 – 10~~ 1–10 commits), the p-value is 0.000, indicating significant difference between SPDX and CycloneDX. However, the effect size (Cliff~~'~~'s delta) is 0.084, which is considered negligible. For the medium commit count group (~~10-100~~ 10–100 commits), the p-value is 0.423, showing no significant difference~~between and . The effect size is −0.031, also negligible. For~~ . Similarly, for the high commit count group (~~100 – 1,000 commits) , the p-value is 0.387, indicating no significant difference between and . The effect size is −0.058, negligible as well. For~~ 100–1,000 commits) and the extremely high commit count group (1,000+ commits), the ~~p-value is~~ p-values are 0.387 and 0.710, ~~showing~~ respectively, both indicating no significant difference~~between and . The effect size is 0.102, negligible.~~ . When considering all commit count groups combined, the p-value is 0.000, ~~indicating~~ showing a significant difference~~between and . However,~~ , but the effect size ~~is~~ remains negligible at 0.116~~, which is still negligible~~ .

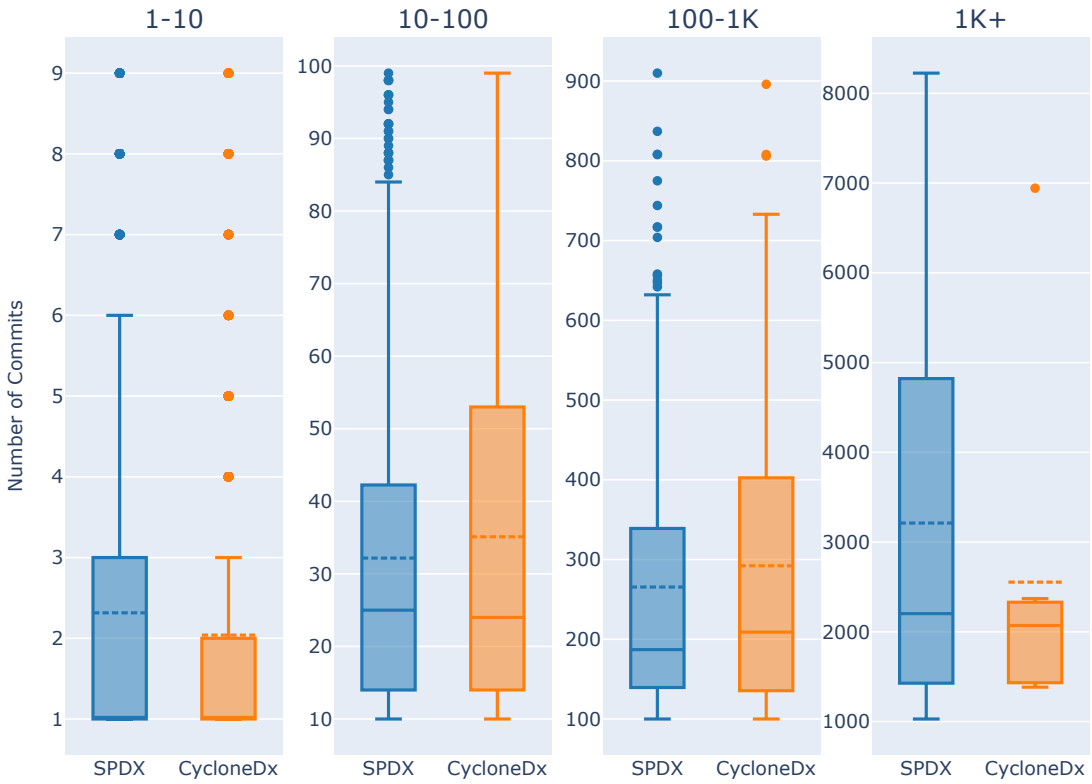


Fig. 8: Distribution of open-source contributor groups (by number of normalized commits) across 470 SPDX and 171 CycloneDX tool repositories.

To further illustrate these findings, Figure 8 shows the distribution of open-source contributors across various commit count groups for SPDX and CycloneDX tool repositories. Compared with the two, the median and mean number of commits for SPDX (1 and 2.32) and CycloneDX (1 and 2.04) are similar for small commit counts ($1 - 10$), indicating a slightly higher commitment from returning contributors in SPDX. Similarly, for medium commit counts ($10 - 100$), both SPDX (median: 25, mean: 32.18) and CycloneDX (median: 24, mean: 35.11) exhibit a comparable level of commitment among returning contributors. However, for high counts of commits ($100 - 1k$), CycloneDX (median: 209, mean: 292.18) shows a lower level of commitment than SPDX (median: 187, mean: 265.52). Finally, for the highest commit counts ($1k+$), SPDX shows a higher commitment level (median: $2,203$, mean: $3,212.05$) than CycloneDX (median: $2,071$, mean: $2,554$).

RQ2: Returning Contributors

From the outlook, SPDX tools, when compared to CycloneDX tools, attract a higher level of commitment from its contributors with $1,000+$ commits, and similarly, attract a slightly higher level of commitment from contributors with a small number of commits ($1 - 10$ commits). In contrast, CycloneDX show more returning contributors in the range of $10 - 1000$ commits. However, despite a statistically significant difference in some cases, the number of returning contributor remain negligible across all commit count groups in tools of both standards.

4.3 Issues in the SBOM OSS ~~tools~~ Tools (RQ3)

Since previous studies have ~~shown~~ demonstrated the potential of ~~uncovering~~ mining GitHub issues to uncover developers' concerns and challenges~~from mining Github issues~~ (Kikas et al., 2015; Batoun et al., 2023), ~~in this RQ , we study~~ this RQ examines the prevalence of GitHub issues in the open-source SBOM ~~tools~~ ecosystem that support the CycloneDX and SPDX ~~formats~~standards. Issues in ~~Github repositories are used~~ GitHub repositories serve as a means to report, discuss, and track ~~work within a repository. Knowing~~ various aspects of development. Understanding the resolution time for ~~a specific issue category would draw the attention of SBOM developers to the tools that need improvement. Furthermore~~different issue categories can shed light on the complexity of specific SBOM-related challenges, helping to identify particularly difficult use cases and ecosystems that require further attention. Additionally, insights into the ~~kind of issues will enable SBOM practitioners to make~~ nature of these issues can aid SBOM practitioners in navigating common pain points and making informed decisions about ~~tool selection and implementation in their supply chain. Furthermore, developers can use this feedback to prioritize bug fixes, address user needs, and enhance the overall functionality of SBOM tools~~adoption and implementation within their supply chains. Moreover, a deeper understanding of issue patterns would allow developers to address recurring challenges, ultimately contributing to a more robust and sustainable SBOM ecosystem.

4.3.1 Comparing closed and open issues.

CycloneDX tools have a higher mean number of ~~mean~~ issues than SPDX tools. ~~Number~~ The number of issues, whether open or closed, can indicate the level of activity, responsiveness to bug reports or feature requests, and overall project health (Kikas et al., 2015). Additionally, the more a project is used, the more issues are likely to be reported. We compare the CycloneDX and SPDX tool repositories by the number of normalized issues using the Mann-Whitney U test and Bonferroni correction applied, and find a statistically significant difference between the tools of both ~~formats, with p-value=0.0003~~ standards, with $p = 0.0003$ and Cliff's Delta effect size of 0.618 (Large). For a fair comparison, in this RQ, we consider only those issues that were created after 2018 because CycloneDX technology was released in May 2017. We find that ~~tools have a higher mean average for having number~~

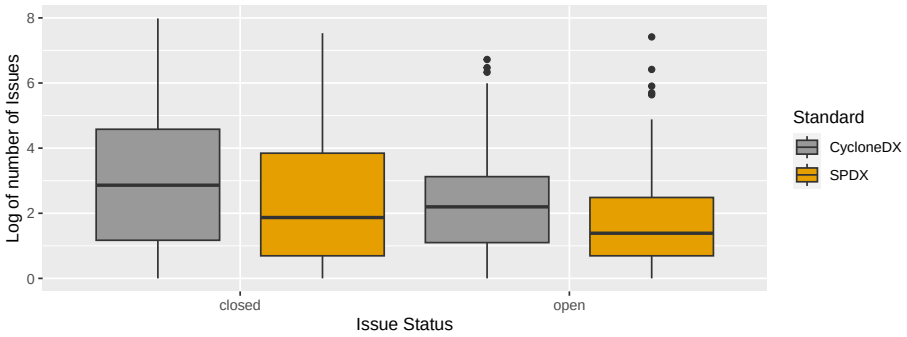


Fig. 9: Representation of 17,093 issues in 90 CycloneDX tool repositories and 16,747 issues in 171 SPDX tool repositories.

~~of issues than tools, in both open and closed issue categories.~~ **CycloneDXtools have a higher mean average [redacted] of normalized issues than SPDXtools, in both open and closed issue categories.** Furthermore, after applying a 1000 samples bootstrapping with a 95% confidence interval, we still find that CycloneDX have more mean number of normalized issues (lower bound:53.9 and upper bound: 15[illegible]) than SPDX (lower bound:34.01 and upper bound: 85.89).

CycloneDX tools have a higher mean number of open and closed ~~mean~~ issues than SPDX tools. We then group all the issues by their status: open and closed. On the issue status level, Figure 9 shows 4,974 open issues in 171 SPDX tools compared to 4,449 open issues in 90 CycloneDX tools. The figure also shows 11,773 closed issues in SPDX tools compared to 12,644 closed issues in CycloneDX tools. We apply the Mann-Whitney U test with Bonferroni correction to compare the tools by number of normalized open and closed issues. In open issues, by prevalence, we found a statistical difference with ~~a p-value=0.0039~~ $p = 0.0039$ with Cliff's Delta effect size 0.586 (Large), and in closed issues we find a significant difference with ~~a p-value=0.0207~~ $p = 0.0207$ with Cliff's Delta effect size 0.572 (Large). We find that the mean number of normalized open issues in CycloneDX is 54.25 with a standard deviation of 141.11, which surpasses the mean number of SPDX ~~open issues~~ 's normalized open issues (38.55) with a standard deviation of 161.87. Similarly, the mean number of normalized closed issues in CycloneDX is 162.10 with a standard deviation of 441.07 and the mean number of normalized closed issues in SPDX is 85.31 with a standard deviation of 258.46. A higher standard deviation of SPDX tools in both open and closed issues suggests inconsistency in issue activity in SPDX tools compared to CycloneDX.

RQ3: Issue Activity

CycloneDX tools community is more active in terms of issue creation ([illegible]96% more open issues than SPDX tools) and resolution (92.50% more closed issues than SPDX tools) than SPDX, with a significant difference and large effect size.

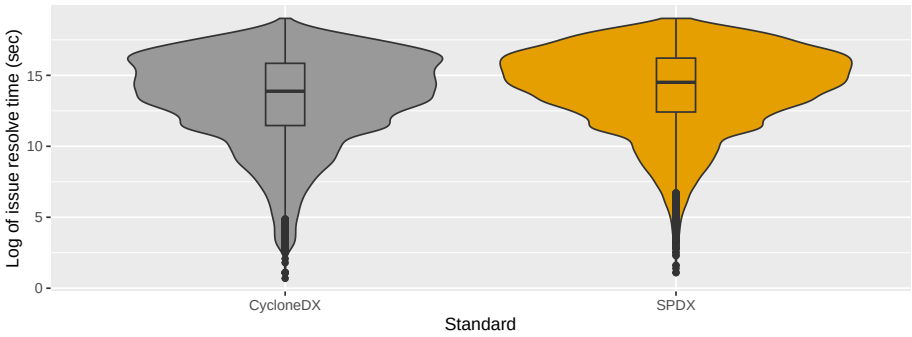


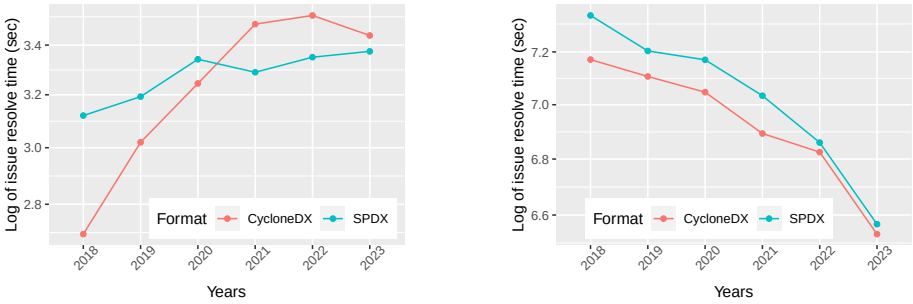
Fig. 10: Resolution time of 13,281 closed issues in 90 CycloneDX tool repositories and 13,352 closed issues in 191 SPDX tool repositories.

4.3.2 Comparing resolution time of the issues.

CycloneDX tools have a faster and more stable resolution time for their Github issues than SPDX. We investigate if the tools of one ~~format~~ standard have a faster response time for resolving the issues when compared to the tools of another ~~format~~standard. To compare the resolution time ~~of the issues across the two formats~~across CycloneDX~~and~~ SPDX , we apply the Mann-Whitney U test with Bonferroni correction and ~~find that both formats' issue resolution time significantly differ from eachother with $p - value = 1.32e - 60$~~find a significant difference between the two ($p = 1.32e^{-60}$). Figure 10 presents the ~~resolve time of both formats' tools, we observe that, in terms of mean resolution time, despite having more~~ resolution times, showing that despite having a higher number of issues than SPDX, CycloneDX ~~still has 67.69% better resolution timethan .~~ tools still achieve a 67.69% lower resolution time. On average, SPDX ~~has an averageresolution time of~~ issues take 130.17 days ~~, while~~ to resolve, whereas CycloneDX ~~has an average resolution time of~~ issues are resolved in 88.12 days. ~~The~~ Additionally, the standard deviation of SPDX ~~tools~~ issue resolution time is ~~also 71% higher than~~ 71% higher than that of CycloneDX ~~. Having better resolution time and lesser standard deviation suggests that~~ further suggesting that the CycloneDX OSS tools community is more ~~active towards resolving the Github~~ efficient and responsive in handling issues (Kikas et al., 2015). In our analysis, we use mean resolution time instead of median to align with prior work that examines issues and defect resolution time in software projects (Assar et al., 2016; Herbsleb et al., 2001).

4.3.3 ~~Trend analysis of the issues.~~

~~**tools have more issues and better resolution time since 2021.** Since there is a significant difference in the overall~~ Given the significant disparity in the number of OSS tool issues ~~in~~ between SPDX and CycloneDX, we further ~~compare them to analyze the recent trend, i.e., of the past few years (~~examine recent trends from 2018 to ~~2023).~~ 2023. Figure 11 ~~suggests~~ indicates that until 2020~~was having more number of issues~~, ~~however,~~ SPDXsaw a higher number of reported issues, although CycloneDX ~~always had a better~~ consistently maintained a lower resolution time.



(a) Prevalence of issues. (b) Resolution time of issues.

Fig. 11: Trend of prevalence and resolution time of issues from 2018 to 2023.

~~Since after 2020,~~ However, **after 2020, the CycloneDXcommunity became more active than SPDX, handling a greater volume of issues while maintaining lower resolution times.** Notably, since 2021—the year the U.S. government issued an executive order mandating the adoption of Software Bill of Materials (SBOMs) for companies conducting business with the U.S. (Remaley and Remaley, 2021) –the CycloneDX ~~community has been more active with more number of issuesand better resolution times~~standard has demonstrated heightened engagement in resolving issues, further solidifying its ecosystem.

RQ3: Issue Activity

CycloneDX tools resolve issues 67.69% faster than SPDX, with a 71% lower standard deviation in resolution time. Furthermore, the CycloneDX community has become more active than SPDX, particularly since 2021, addressing more issues with faster resolution times.

4.3.3 Tags analysis of the issues.

SPDX tools have most of the issues related to "Feature Development and Enhancement" and "Code Components", while CycloneDX tools have more issues related to "Bug Fixes and Defects"~~and "Feature Development and Enhancement"~~. Understanding the type of issues in each SBOM ~~format~~ standard would help tool developers improve their tools and help SBOM users make the right decision when selecting an SBOM tool for their project. We identify 14 distinct categories from the issues collected from the 87 SPDX and 61 CycloneDX tool repositories, using the manual tag classification approach described in Section 3.4. We have listed the indices and names of each category, accompanied by a short description, the set of relevant tags and one representative example as follows.

C_{01} **Bug Fixes and Defects**
Definition: This category refers to technical problems within the codebase

Tags: exceptions, stale, false-positive, defect, bug, kind/bug, technical issue, stale, good-first-issue, lifecycle/stale, lifecycle/frozen, QA
Example: "PolicyExceptions not being applied to EphemeralContainers" in the "kyverno/kyverno" repository[15].

C_{02} **Code Quality**

Definition: This category refers to improving the overall quality of code.
Tags: refactor, comment-styles, log enhancement, changelog-ignore, package-formats, package scan, performance, validation, generation
Example: "Missing CVEs on openssh" in the "anchore/grype" repository[16].

C_{03} **Community Engagement and Support**

Definition: This category refers to engaging the community, seeking feedback, and providing support to contributors.
Tags: request for comment, upstream, help wanted
Example: "Add bom-refs to OrganizationalEntity and OrganizationalContact" in the "CycloneDX/specification" repository[17].

C_{04} **Configuration**

Definition: This category refers to configuring, packaging, and managing dependencies for the project.
Tags: configuration, docker, helm, installation and packaging, dependencies
Example: "Maximize portability by using fallback pure-python libraries" in the "nexB/scancode-toolkit" repository[18].

C_{05} **Continuous Integration and Deployment**

Definition: This category refers to setting up and maintaining continuous integration and continuous deployment pipelines.
Tags: CI, Type: CI, CI/CD
Example: "Change logo in gh-pages" in the "linterhub/schema" repository[19].

C_{06} **Documentation**

Definition: This category refers to creating and maintaining documentation for the project.
Tags: docs, Documentation, documentation
Example: "review the license type detection" in the "CycloneDX/cyclonedx-node-module" repository[20].

C_{07} **Feature Development and Enhancement**

Definition: This category refers to adding new features and enhancements to the project.
Tags: kind/feature, proposed core enhancement, should have, nice to have, feature, must have, proposal, enhancement, enhancement, new feature, security

[15] `https://github.com/kyverno/kyverno/issues/9484`
[16] `https://github.com/anchore/grype/issues/1695`
[17] `https://github.com/CycloneDX/specification/issues/226`
[18] `https://github.com/nexB/scancode-toolkit/issues/3210`
[19] `https://github.com/linterhub/schema/issues/174`
[20] `https://github.com/CycloneDX/cyclonedx-node-module/issues/234`

Example: "Feature request: Map k8s pod labels as project tags" in the "ckotzbauer/sbom-operator" repository[21].

C_{08} **Integration**

Definition: This category refers to integrating with various operating systems and ecosystems.
Tags: os support, ecosystem:javascript, ecosystem:java, webhook
Example: "scancode-toolkit-mini 30.1.0 - missing dependency" in the "nexB/scancode-toolkit" repository[22].

C_{09} **Libraries**

Definition: This category refers to ~~developing and managing~~ usage of libraries and APIs.
Tags: spdx-utils, API Call, core and api
Example: "How to provide custom license texts?" in the "oss-review-toolkit/ort" repository[23].

C_{10} **Licensing**

Definition: This category refers to copyrights and licensing.
Tags: improve-license-detection, copyright scan, Profile:Licensing, license-review, improve-license-detection, Submit New License, new license/exception: Not Accepted, new license/exception: Accepted, new license/exception request
Example: "New license request: AzFOSS" in the "spdx/license-list-XML" repository[24].

C_{11} **Code Components**

Definition: This category refers to specific components of the codebase, such as serialization, parsers, and package scanning.
Tags: serialization, data model, parser, analyzer, scanner, I/O, package-at-toplevel, package-files, package-formats, package scan, component:dep-sources
Example: "Integrate DWARF plugin into SCTK release and correct documentation" in the "nexB/scancode-toolkit" repository[25].

C_{12} **Release Management and Versioning**

Definition: This category refers to managing releases, handling breaking changes, and versioning of the project.
Tags: breaking change, release-high, pending release, breaking-change, breaking change
Example: "Remove deprecated configuration" in the "anchore/syft" repository[26].

C_{13} **User Interface and Outputs**

Definition: This category refers to the user interface and outputs of the project.
Tags: GUI and outputs, type:cli, end user

[21] https://github.com/ckotzbauer/sbom-operator/issues/172
[22] https://github.com/nexB/scancode-toolkit/issues/2752
[23] https://github.com/oss-review-toolkit/ort/issues/4740
[24] https://github.com/spdx/license-list-XML/issues/1365
[25] https://github.com/nexB/scancode-toolkit/issues/2422
[26] https://github.com/anchore/syft/issues/1864

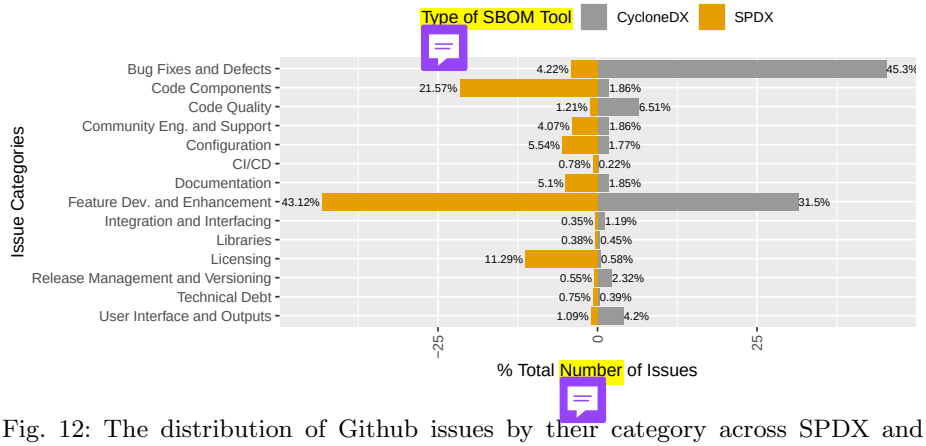


Fig. 12: The distribution of Github issues by their category across SPDX and CycloneDX SBOM tools.

Example: "Incorrect progress bar display" in the "nexB/scancode-toolkit" repository[27].

C_{14} **Technical Debt**
Definition: This category refers to addressing the technical debt of the project. **Tags**: technical-debt, technical debt **Example**: "Refactor analyze_docker_image to reduce complexity" in the "tern-tools/tern" repository[28].

Figure 12 illustrates the prevalence of each issue category. For SPDX, issues related to "Feature Development and Enhancement" are the most prevalent (43.12%), closely followed by "Code Components" (21.57%) and "Licensing" (11.29%). In contrast, CycloneDX exhibits a different distribution, with "Bug Fixes and Defects" being the most prevalent (45.3%), followed by "Feature Development and Enhancement" (31.5%) and "Code Quality" (6.51%). These findings highlight the distinct challenges and priorities that each SBOM ~~format~~ standard faces in its respective tool ecosystems.

Amongst the ~~issue-categories~~issue categories, SPDX tools ~~resolves~~ resolve issues related to Licensing 51.85% faster than CycloneDX, while CycloneDX tools ~~resolves~~ resolve other ~~issue-categories~~ issue categories (~~the ones that have~~ those with significantly different resolution ~~time~~ times in SPDX) 78.87% faster than SPDX. To ~~further understand which format spends how much time resolving what type~~ better understand how each standard handles different types of issues, we ~~determine~~ analyze the resolution time of each issue category ~~in a format, as presented~~ within each standard, as shown in Figure 13. ~~We then~~ Next, we compare the resolution ~~time~~ times of the categories within each ~~format. For this comparison, we apply~~ standard, employing the Mann-Whitney U test ~~of~~ for statistical difference and Cliff's Delta ~~for~~ to assess effect size.

[27] `https://github.com/nexB/scancode-toolkit/issues/389`
[28] `https://github.com/tern-tools/tern/issues/212`

Table 7: Result of Mann-Whitney U Test to compare the difference between the resolution time of issue categories by CycloneDX.vs. SPDX. Better Standard represents the standard with lower resolution time.

Category	**p-value**	**Cliff's delta**	**Effect size**	**Better Standard**
Bug Fixes and Defects	2.2e-16	0.278	Small	CycloneDX
Code Quality	0.257	-	-	-
Code Components	0.8705	-	-	-
Community Engagement and Support	0.1181	-	-	-
Configuration	0.0527	-	-	-
Continuous Integration and Deployment (CI/CD)	0.03588	0.631	Large	CycloneDX
Documentation	0.054	-	-	-
Feature Development and Enhancement	2.2e-16	0.387	Medium	CycloneDX
Integration and Interfacing	0.0088	0.342	Medium	CycloneDX
APIs	0.07501	-	-	-
Licensing	0.0448	0.581	Large	SPDX
Release Management and Versioning	0.3662	-	-	-
Technical Debt	0.4418	-	-	-
User Interface and Outputs	0.0077	0.4008	Medium	CycloneDX

~~We find significant difference in the following categories: "Bug Fixes and Defects" with small effect size, "Continuous Integration and Deployment (CI/CD)" with large effect size, "Licensing" with large effect size, "Integration and Interfacing" with medium effect size, "User Interface and Outputs " with medium effect size, and "Feature Development and Enhancement" with medium effect size.~~ The results of ~~the statistical tests can be observed~~ these statistical tests are presented in Table 7. It shows that CycloneDX resolves issues 174.03% faster in the "Bug Fixes and Defects" category, 15.9% faster ~~in the~~ "Continuous Integration and Deployment (CI/CD)", 73.41% faster ~~in the~~ "Feature Development and Enhancement", 46.52% faster ~~in the~~ "Integration and Interfacing", and 64.47% faster ~~in the~~ "User Interface and Outputs~~" category.Whereas~~." In contrast, SPDX resolves issues ~~that are~~ related to "Licensing" 51.85% faster than CycloneDX , which is reasonable given that Licensing was its main focus when SPDXwas first developed. Other use cases and issue categories were later integrated, highlighting how SPDX's optimization for licensing issues translates into faster resolution times in this category. This underscores the significant difference in focus and development priorities between CycloneDXand SPDX .

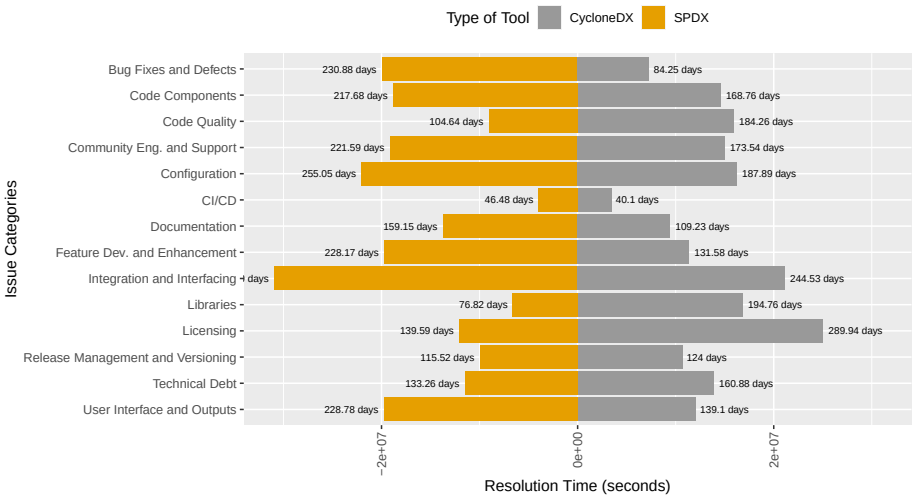


Fig. 13: Distribution of resolution time (in seconds) of issue categories of SPDX and CycloneDX tools.

RQ3: Most prevalent issues

SPDX tools have most of the issues related to "Feature Development and Enhancement", while CycloneDX tools have more issues related to "Bug Fixes and Defects". Among the categories that have significantly different resolution times, SPDX tools resolves issues related to Licensing 51.85% faster than CycloneDX, while CycloneDX tools resolves the remaining issue-categories 78.87% faster than SPDX.

4.4 Prevalence of SBOM tools in OSS projects (RQ4)

In this RQ, we investigate ~~those Github projects that carry the create command of an SBOM tool in their build configuration or code~~GitHub projects that use the CycloneDX~~or~~ SPDX~~standard~~ for their project's SBOM. By analyzing these open-source projects, we provide insights into the developer community trends and the characteristics of the projects that use SBOM tools (SPDX or CycloneDX). We further calculate the demand of SBOM features in the open-source community. SPDX and CycloneDX tool developers may use this information to improve the features of their tools.

4.4.1 Insights on the developer community of OSS projects that use SBOM tools.

To understand the significance of ~~the~~ projects that use the SPDX and CycloneDX tools, we assess the community health of ~~the identified projects through~~ identified

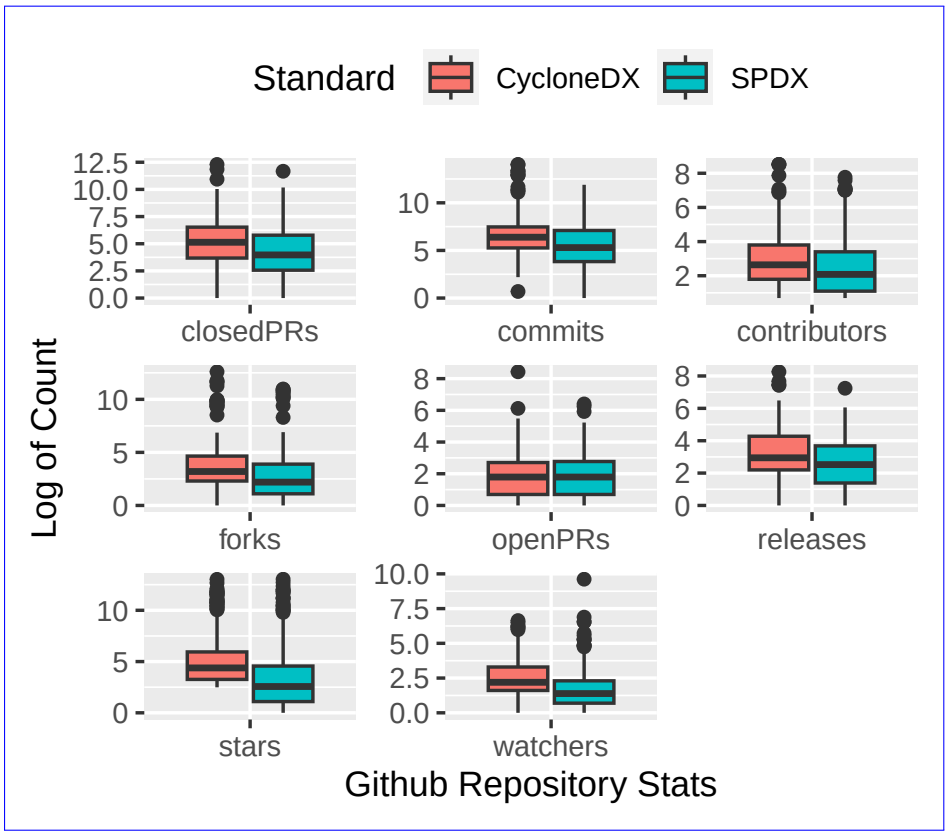


Fig. 14: Community normalized health metrics of Top-250 projects that use SPDXtools .vs. Top-250 projects that use CycloneDXtools. Log of count is normalized by the age of repository in days × 100.

projects using the same metrics that were used in RQ2 – # number of Stars, Watchers, Forks, Issues, Contributors, Pull Requests, Releases, and Commits. These metrics have previously been used in literature to assess many quality factors of software repositories, such as consistent documentation, number of downloads, popularity, and development activity Zerouali et al. (2019). These metrics also provide an idea about the health of the open-source community around these projects C. P. L. Foundation (Accessed May 27, 2024).

GitHub projects using CycloneDX outperforms outperform those using SPDX in all the open-source community health factors, such as number of stars, watchers, forks, PRs, releases, contributors, and commits. Figure 14 presents the values of the metrics of the project repositories that employ CycloneDX or SPDX tools. To compare the difference between the SPDX-using and CycloneDX-using projects, we applied the Mann Whitney-U test (Wilcoxon et al., 1970) followed by the Bonferroni correction, on the non-normal distributions of the extracted metric values. Table 8 shows test results and the respective effect sizes(Cohen, 1992). It can be observed from this table that

Table 8: Mann-Whitney U Test (Wilcoxon et al., 1970) applied to evaluate the difference between the project characteristics and popularity metrics of 250 CycloneDX employing projects and 250 SPDX employing projects. $p - value < 0.05$ indicates a significant difference between the two distributions.

Metric	p-value	Cliff's Delta	Effect size
ClosedIssues	3.79^{-12}	0.666	Large
ClosedPullRequest	2.74^{-09}	0.655	Large
Contributors	0.07^{-01}	0.576	Large
Forks	2.50^{-16}	0.709	Large
OpenIssues	5.39^{-09}	0.618	Large
OpenPullRequest	0.03^{-02}	0.525	Large
Releases	0.43^{-01}	0.592	Large
Stars	6.09^{-27}	0.783	Large
Watchers	3.35^{-12}	0.668	Large
Commits	3.38^{-06}	0.632	Large

both ~~the~~ sets are significantly different from each other for all the metrics, with a "large" effect size too.

Projects using CycloneDX tools have a higher median value (normalized by age of repositories in days) in terms of stars (0.80 in CycloneDX vs. only 0.04 in SPDX), forks (0.22 in CycloneDX vs. only 0.04 in SPDX), and watchers (0.08 in CycloneDX.vs. 0.033 in SPDX). ~~Number of stars (also known as stargazers) is considered an indicator of admiration and a positive attitude towards the chosen repository, which~~ CycloneDX-using projects higher number of stars and forks can indicate higher trust and usage ~~Borges and Valente (2018); and adopting projects beats adopting projects in this metric~~from the OSS community Borges and Valente (2018); Jiang et al. (2017) . Furthermore, CycloneDX-using projects ~~having a higher number of forks indicating an active contributing community compared to -using projects (Jiang et al., 2017) and -using projects~~ have a higher number of watchers ~~indicating~~ that could indicate greater observability by the OSS community compared to the ~~-using projects (Borges and Valente, 2018)~~ SPDX-using projects Borges and Valente (2018).

Repositories using CycloneDX tools ~~have a notably~~ exhibit significantly higher issue activity ~~both in terms of median (normalized by age of repositories~~ compared to those using SPDXtools, as indicated by median values normalized by repository age (in days)~~open pull request~~ . Specifically, CycloneDXrepositories show higher median open pull requests (0.03 in CycloneDX vs. 0.01 in SPDX), median closed pull ~~request~~ requests (1.57 in CycloneDX vs. 0.21 in SPDX), median releases (0.19 in CycloneDX vs. 0.12 in SPDX), median contributors (0.14 in CycloneDX vs. 0.08 in SPDX), and median commits (5.94 in CycloneDX vs. 2.00 in SPDX). This trend suggests that CycloneDX ~~projects receive~~ -based projects attract more attention and effort from the OSS community. ~~In addition, greater attention to~~ One potential explanation for this higher activity could be that CycloneDXtools enable more streamlined collaboration and integration, fostering greater involvement from contributors. Additionally, the higher frequency of pull requests and commits ~~reflects the contribution of OSS to projects~~ [redacted]

RQ4: Community-health of projects that use SBOM tools

All the community-health metrics suggest that Top-250 CycloneDX-using projects have a healthier OSS community than Top-250 SPDX-using projects, with a large effect size on the difference between the two standards.

4.5 Insights on the language distribution of OSS projects that use SBOM tools.

Projects written in procedural languages prefer SPDX tools, while projects written in scripting languages prefer CycloneDX tools. The choice of the programming language for a project significantly impacts development efficiency, performance, maintainability (Bhattacharya and Neamtiu, 2011), security, and code quality (Ray et al., 2017). To understand which programming language is preferred by which project (CycloneDX-using or SPDX-using), we classify the projects based on programming paradigm, i.e., whether a project is written in a procedural, functional, or scripting language.

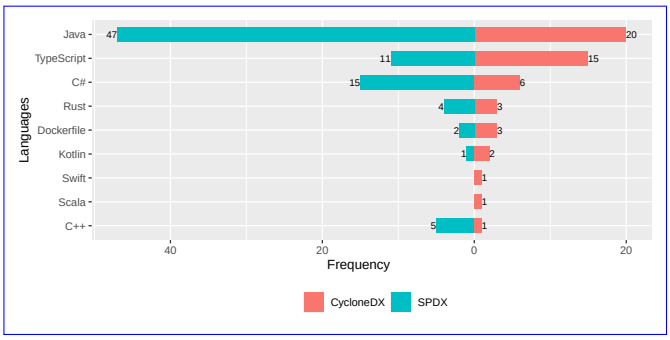
Figure 15 shows the language distribution of projects ~~into~~ using procedural and scripting languages. ~~In , it~~ It can be observed in this figure that a higher proportion of projects utilizing scripting languages ~~leaning towards~~ favor CycloneDX ~~tools~~-using projects, whereas procedural languages exhibit ~~a greater presence in projectsthat use~~ association to SPDX ~~tools~~-using projects. As ~~a~~ation of procedural languages to SPDX-using projects makes sense because SPDXwas originally developed for the Linux kernel (a C-based project). Procedural languages, including Go, Java, and C#, are commonly used in system-level programming, performance-critical tasks, or the development of large-scale applications, indicating a preference for SPDX tools in such projects. Conversely, scripting languages such as Python, TypeScript, and Shell Scripting are favoured for web development, automation, or data processing, indicating a preference for CycloneDX tools in ~~these~~ such projects.

RQ4: Language preference

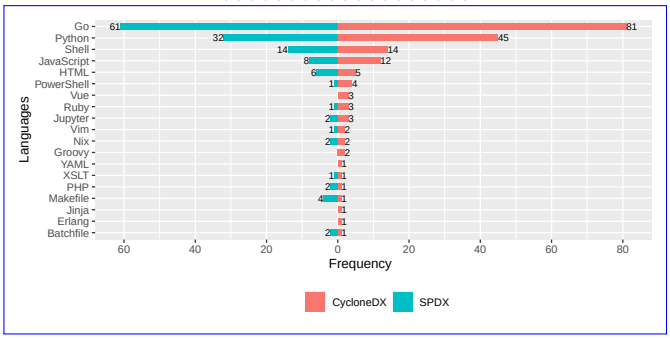
Projects that use CycloneDX tools show a higher usage of scripting languages, while projects that use SPDX tools show a higher usage of procedural languages. This information can help selecting the most suitable tools for a project based on its primary language. It also aids in tailoring training and resources for developers to align with the tools and languages they are most likely to encounter in their projects.

4.6 Tool preference across different SBOM projects

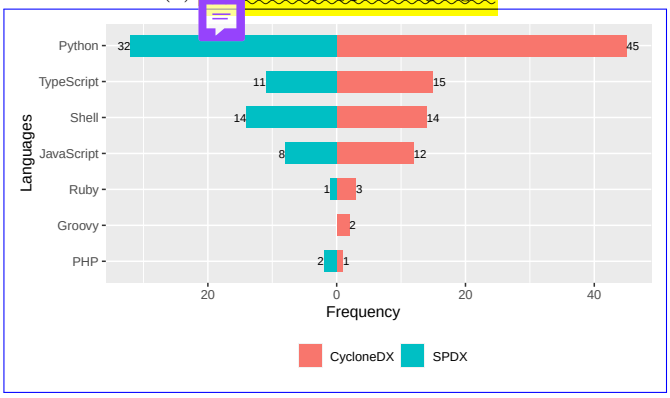
The choice of an SBOM tool varies depending on the number of commits of a project. We determine if different projects prefer using tools that support different ~~usecases~~use cases. For this analysis, we first group the projects



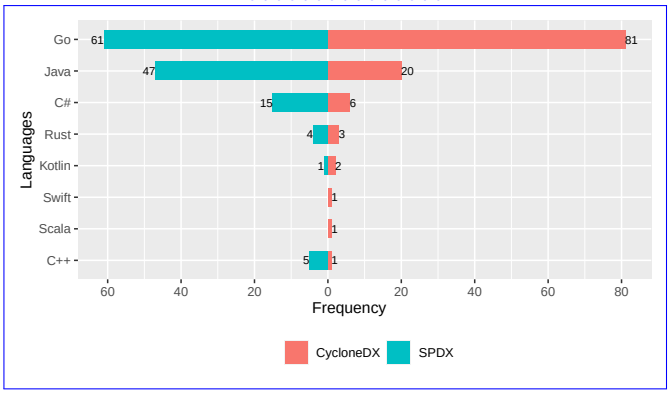
(a) Statically-typed languages



(b) Dynamically-typed languages



(c) Scripting languages



(d) Procedural languages

Fig. 15: Programming language of projects that use CycloneDXtools .vs. projects that use SPDXtools.

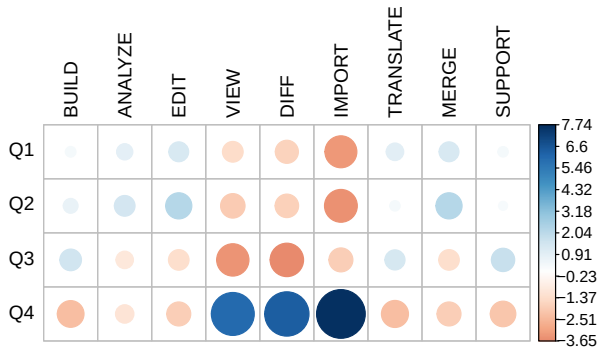


Fig. 16: Visualization of the Pearson residuals to compare the Quartiles (by number of commits) with the ~~usecases~~ use cases of the tools they use. The quartiles are normalized by the age of repository in days. Q1 = 0.00029–0.042, Q2 = 0.043–0.16, Q3 = 0.17–0.055, Q4 = 0.56–2508.56. The size of the bubble denotes the significance of a particular ~~usecase~~ use case towards the chi-square result.

into four quartiles based on their number of contributors. We applied Chi-square and found that there is a statistical difference among the quartiles ~~wtih p-value = $2.29e^{-20}$~~($p = 2.29e^{-20}$), however, with a weak effect (~~Cramér's V = 0.095~~*Cramér's V* = 0.095). We then group the projects into four quartiles based on their number of commits and find a statistical difference among the quartiles (~~p-value = $1.65e^{-40}$~~$p = 1.65e^{-40}$) with a moderate effect size (~~Cramér's V = 0.124~~*Cramér's V* = 0.124) Cramér (1999). We present the distribution of chi-square residual values in Figure 16. Lower values in this figure indicate lesser demand for a ~~usecase~~use case, while higher ~~value~~ values indicate higher demand for a ~~usecase~~use case. The size of the bubble represents the proportion of a quartile ~~usecase~~ use case that contributed towards the chi-square's significance.

Our analysis reveals that *View*, *Diff*, and *Import* usecases from the *Consume* category of the NTIA' Tools Usecase Taxonomy exhibit larger Pearson residuals for the last two quartiles (Q3 and Q4), suggesting that projects having larger number of commits prefer tools ~~with *View*, *Diff*, and *Import*~~ ~~usecases~~ that support all the usecases of the *Consume* category. Furthermore, *Analyze* ~~,~~ and *Edit* from the *Produce* category, and *Merge* from the *Transform* category are more prevalent in the first two quartiles (Q1 and Q2), suggesting that projects having lesser number of commits prefer tools with ~~*Analyze*, *Edit*, and *Merge*~~ ~~usecases~~these use cases.

RQ4: Tool preference across different SBOM projects

Large projects (by number of commits) prefer tools that support *View*, *Diff*, and *Import* use cases, while smaller projects have lesser demand for such tools. Similarly, smaller projects are more inclined towards tools that support *Edit*, and *Merge*, while larger projects have lesser demand for such tools.

5 ~~Threats to Validity~~Implications

5.1 ~~Conclusion Validity~~

Improvement of Open-Source SBOM Tool ecosystem. The study sheds light on the strengths and limitations of the open-source SBOM tool ecosystem compared to proprietary ones. Our findings indicate that open-source tools underperform particularly in the *Analyze* and *View* use cases, where proprietary tools exhibit stronger support. To address this gap, open-source developers should focus on improving visualization capabilities for SBOMs and enhancing analytical features, thereby bridging the gap between open-source and proprietary tools and making open-source options more versatile and competitive across various applications.

~~This study focuses on the tool ecosystem of the two most popular SBOM formats,~~

Enhanced Decision-Making for SBOM Tool Adoption. The insights from this study provide practitioners and policymakers with a clearer understanding of the strengths and weaknesses of different SBOM tools. This enables more informed decision-making when selecting tools, such as choosing based on integration with build processes or support for multiple standards. These findings can guide organizations in adopting the most suitable tools for their unique needs, facilitating strategic planning and efficient adoption within the software supply chain.

Identification of Prevalent Issues for Targeted Solutions. The analysis of commonly reported issues in open-source SBOM tools offers a detailed understanding of the specific challenges faced by tools in different technologies. By categorizing these issues and comparing the resolution times between SPDX and CycloneDX ~~. As such, our findings are specific to these formats and cannot be generalized to other SBOM technologies, such as Software Identification Tags (SWID) (NTIA, 2021b). Future research could explore the performance and~~ tools, the study highlights key areas that developers can address to improve tool performance. For example, since CycloneDXtools show a consistently longer resolution time for issues related to license compliance compared to CycloneDX, developers could prioritize fixing these compliance issues. This targeted approach to problem-solving can significantly enhance the usability and reliability of the SBOM tools' ecosystem, ultimately leading to more secure and efficient software supply chains.

Enhancement of Community Health and Engagement. Evaluating the community health of ~~other SBOM formats and tools to provide a more comprehensive understanding of the broader SBOM ecosystem.~~ open-source SBOM tools using CHAOSS metrics helps identify which tools are more actively maintained and adopted. This information guides open-source developers to contribute to high-demand tools, improving their functionality and user support. Moreover, it assists practitioners in selecting tools that are actively developed and maintained, ensuring long-term sustainability and reliability within their organizations.

6 Threats to Validity

6.1 Construct Validity

While the manual tag classification approach provides valuable insights, it may introduce threats to validity, particularly due to the open card sorting process. Some issues naturally span multiple tags, leading to potential biases in classification. However, we mitigate this by validating our labels through iterative negotiation, which has resulted in a high level of inter-rater agreement.

In RQ4, we did not configure or execute the GitHub projects to verify the generation of SBOM files. Instead, we focused on identifying the presence of SBOM generation patterns in their code or build configuration files. This approach allowed us to efficiently analyze a large number of projects, but did not account for the actual implementation or effectiveness of these patterns. Additionally, our study on the open-source projects that use the technologies does not investigate the real world adoption or usage of SPDX and CycloneDX ~~formats~~ standards in the industry, which could offer further insights into their practical importance and influence. Future research may consider addressing these aspects to provide a more comprehensive understanding of the impact and adoption of various SBOM ~~formats~~standards.

While there is another SBOM ~~format~~ standard called SWID, we chose not to include it in our study, as it is considered more a software identifier than a comprehensive SBOM ~~format~~standard, such as SPDX and CycloneDX (Brudo, 2024). SWID tags provide a simple method to track software inventory, but lack the depth and analysis capabilities offered by SPDX and CycloneDX, which include detailed information on software components, licenses, vulnerabilities, and other metadata (Das, 2023). Comparison of SWID tool ecosystems with SPDX and CycloneDX would not yield equivalent insights due to the fundamental differences in their scope and purpose.

6.2 Internal Validity

SPDX and CycloneDX support various file ~~formats~~standards, but only JSON and XML are common to both. Although each ~~format~~ standard could potentially cover different SBOM documentation aspects such as licensing and security, we conducted a manual ad hoc comparison of the JSON and XML specifications provided by each technology. Our analysis suggested that both JSON and XML specifications cover the same SBOM documentation features. Based on this finding and for simplicity, we decided to focus solely on the XML specification for the RQ1 analysis. We believe that this decision does not compromise the validity of our results, as both file ~~formats~~ standards appear to support equivalent features of the SBOM documentation. Future research could explore the potential differences between file ~~formats~~ standards in more depth to validate this assumption.

6.3 External Validity

This study focuses on the tool ecosystem of the ~~two~~ most popular SBOM standards, SPDX~~and~~ CycloneDX. While our findings are ~~spe~~ific to these standards, both are part ~~of~~ the NTIA's recommended SBOM technologies. Therefore, although the findings may not directly generalize to other SBOM technologies, such as Software Identification Tags (SWID) (NTIA, 2021b), future research could examine the performance and community health of other SBOM standards and tools to provide a more comprehensive understanding of the broader SBOM ecosystem.

In RQ3, our analysis ~~relies on the~~ is based on issues reported in the GitHub repositories of the CycloneDX and SPDX tools. ~~As a result, the~~ This means that our findings may not ~~apply~~ be fully applicable to other SBOM ~~formats or~~ standards or to repositories hosted on platforms other than GitHub. To ~~minimize~~ address this limitation, we focus on the two most ~~popular SBOM technologieswith a substantial~~ widely used SBOM technologies, which have a large and active user base, ~~ensuring~~ providing a broad and representative view of their ecosystems. However, it is important to note that our study does not capture ~~all~~ the CycloneDX and SPDX-related discussions, issues, or contributions ~~that occur~~ outside of GitHub. While we believe our approach ~~provides~~ offers valuable insights, future research could explore additional sources ~~of information to further enrich~~ to further enhance our understanding of these ecosystems.

Focusing on tools available on the official CycloneDX and SPDX websites aimed to streamline our data collection process by leveraging expert-curated lists. This approach increases the likelihood of identifying functional tools, as they have been vetted by the respective communities. Additionally, it reduces the risk of including prototype tools that may not be suitable for practical use. However, this strategy may inadvertently exclude functional, non-prototype tools hosted elsewhere. While the likelihood of this being relatively low, future research could expand the scope to include a broader range of sources, ensuring a more comprehensive overview of the available tools. Nonetheless, our current approach offers a solid foundation for understanding the landscape of CycloneDX and SPDX tools.

Our analysis is based on a specific point in time snapshot of the SPDX and CycloneDX ecosystems. As these ecosystems continue to grow and evolve, the distribution of tools and issue categories can change, and new trends can surface. While our findings provide valuable insights into the current state of these ecosystems, ongoing monitoring and evaluation will be necessary to capture emerging trends and updates in the dynamic ecosystem of SPDX and CycloneDX tools.

7 Conclusion

This study provides a comprehensive analysis of the SBOM tools ecosystem, comparing the two predominant SBOM ~~formats~~standards: SPDX and CycloneDX. Through qualitative and quantitative analyses of 108 open-source and 62 proprietary SBOM tools, as well as an in-depth investigation of the community health metrics of the 641 tools and the characteristics of top open-source projects, we have obtained several key insights.

Firstly, we observed that proprietary tools generally offer a broader range of use cases compared to open-source SBOM tools, particularly in the areas of build automation, artifact analysis, and visualization. This indicates a need for the open-source community to enhance the functionalities of their SBOM tools to bridge this

gap. Secondly, our findings on community health metrics reveal that CycloneDX tools tend to have higher developer engagement, as evidenced by their superior value in stars, watchers, forks, pull requests, and releases. [redacted] Moreover, the analysis of GitHub issues highlighted the prevalent challenges faced by both the SPDX and CycloneDX tools. While the CycloneDX tools showed a faster issue resolution time, the SPDX tools excelled in areas, such as community engagement and support. This information is crucial for developers and contributors to focus their efforts on addressing the specific needs and challenges associated with each ~~format~~standard. Finally, our study of GitHub projects that initialize SBOM tools in their code showed distinct preferences and adoption patterns. Projects using CycloneDX tools demonstrated better health metrics, with a higher number of stars, watchers, releases, pull requests, and contributors. Furthermore, the study found that dependency projects with varying levels of recurring contributions exhibit distinct preferences for SBOM operations.

In conclusion, while both SPDX and CycloneDX ~~formats have their unique strengths and weaknesses~~standards have distinct advantages and challenges, CycloneDX tools ~~appear to be more popular and actively maintained within the open source community.~~ tend to have broader adoption and greater ongoing development within the open-source community. This observation suggests the possibility of factors such as resource availability, tool maturity, or simplicity of the CycloneDXstandard influencing its activity. For practitioners and policymakers, this study ~~provides valuable~~ offers important insights into the current ~~state~~ landscape of SBOM tools, ~~guiding them in making informed decisions regarding~~ helping [redacted] informed decision-making about tool selection and implementation to ~~improve~~ strengthen software supply chain security. We ~~identify areas for improvement for~~ also highlight areas where SBOM tool developers ~~and offer~~ can improve, providing a roadmap to enhance ~~the functionalities~~ both the functionality of their tools and ~~community support , ultimately contributing to the development of a robust SBOM tools~~ the support of their respective communities, thereby fostering the growth of a more robust SBOM ecosystem.

References

Assar S, Borg M, Pfahl D (2016) Using text clustering to predict defect resolution time: a conceptual replication and an evaluation of prediction accuracy. Empirical Software Engineering 21:1437–1475

Balliu M, Baudry B, Bobadilla S, Ekstedt M, Monperrus M, Ron J, Sharma A, Skoglund G, Soto-Valero C, Wittlinger M (2023) Challenges of producing software bill of materials for java. `2303.11102`

Batoun MA, Yung KL, Tian Y, Sayagh M (2023) An empirical study on github pull requests' reactions. ACM Transactions on Software Engineering and Methodology

Berger A (2023) What is Log4Shell? The Log4j vulnerability explained (and what to do about it) — dynatrace.com. `https://www.dynatrace.com/news/blog/what-is-log4shell/`, log4j vulnerability explained

Bhattacharya P, Neamtiu I (2011) Assessing programming language impact on development and maintenance: A study on c and c++. In: Proceedings of the 33rd International Conference on Software Engineering, Association for Computing Machinery, New York, NY, USA, ICSE '11, p 171–180, DOI 10.1145/1985793.1985817, URL https://doi.org/10.1145/1985793.1985817

Bi T, Xia B, Xing Z, Lu Q, Zhu L (2023) On the way to sboms: Investigating design issues and solutions in practice. ACM Transactions on Software Engineering and Methodology

Borges H, Valente MT (2018) What's in a github star? understanding repository starring practices in a social coding platform. Journal of Systems and Software 146:112–129

Brudo B (2023) What Happens When an AI Company Falls Victim to a Software Supply Chain Vulnerability. https://scribesecurity.com, [Victim to a Software Supply Chain Vulnerability]

Brudo B (2024) Spdx vs. cyclonedx. https://scribesecurity.com/blog/spdx-vs-cyclonedx-sbom-formats-compared/, [What are SBOM Formats]

C P L Foundation (Accessed May 27, 2024) Community health analytics open source software. https://chaoss.community/kb-metrics-and-metrics-models/

Campbell JL, Quincy C, Osserman J, Pedersen OK (2013) Coding in-depth semistructured interviews: Problems of unitization and intercoder reliability and agreement. Sociological methods & research 42(3):294–320

Chopra A, Mo M, Dodson S, Beschastnikh I, Fels SS, Yoon D (2021) "@ alex, this fixes# 9": Analysis of referencing patterns in pull request discussions. Proceedings of the ACM on human-computer interaction 5(CSCW2):1–25

Cohen J (1992) A power primer. Tutorials in Quantitative Methods for Psychology 112, DOI 10.1037/0033-2909.112.1.155

Cox R (2019) Surviving software dependencies. ACM New York, NY, USA, vol 62, pp 36–43

Cramér H (1999) Mathematical methods of statistics, vol 43. Princeton university press

Das R (2023) Sbom formats compared. https://www.techtarget.com/searchsecurity/tip/SBOM-formats-compared-CycloneDX-vs-SPDX-vs-SWID-Tags, [CycloneDX vs. SPDX vs. SWID Tags]

Foundjem A, Eghan E, Adams B (2021) Onboarding vs. diversity, productivity and quality—empirical study of the openstack ecosystem. In: 2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE), IEEE, pp 1033–1045

Gisev N, Bell JS, Chen TF (2013) Interrater agreement and interrater reliability: key concepts, approaches, and applications. Research in Social and Administrative Pharmacy 9(3):330–338

Gkortzis A, Feitosa D, Spinellis D (2021) Software reuse cuts both ways: An empirical analysis of its relationship with security vulnerabilities. Elsevier, vol 172, p 110653

Hendrick S (2022) Software bill of materials (sbom)and cybersecurity readiness. The Linux Foundation

Herbsleb JD, Mockus A, Finholt TA, Grinter RE (2001) An empirical study of global software development: distance and speed. In: Proceedings of the 23rd International Conference on Software Engineering. ICSE 2001, IEEE, pp 81–90

ISO/IEC (2022) Swid. https://wwwisoorg/standard/45170html ISO/IEC JTC 1/SC 27

Jarczyk O, Gruszka B, Jaroszewicz S, Bukowski L, Wierzbicki A (2014) Github projects. quality analysis of open-source software. pp 80–94, DOI 10.1007/978-3-319-13734-6_6

Jiang J, Lo D, He J, Xia X, Kochhar PS, Zhang L (2017) Why and how developers fork what from whom in github. Empirical Software Engineering 22, DOI 10.1007/s10664-016-9436-6

Jiao J, Tseng MM, Ma Q, Zou Y (2000) Generic bill-of-materials-and-operations for high-variety production management. Concurrent Engineering 8(4):297–321, DOI 10.1177/1063293X0000800404

Kampenes VB, Dybå T, Hannay JE, Sjøberg DI (2007) A systematic review of effect size in software engineering experiments. Information and Software Technology 49(11-12):1073–1086

Kikas R, Dumas M, Pfahl D (2015) Issue dynamics in github projects. In: Product-Focused Software Process Improvement: 16th International Conference, PROFES 2015, Bolzano, Italy, December 2-4, 2015, Proceedings 16, Springer, pp 295–310

Ladisa P, Plate H, Martinez M, Barais O (2022) Taxonomy of attacks on open-source software supply chains

Linux (2023) Spdx. https://spdxorg/

Martin RA (2020) Visibility & control: Addressing supply chain challenges to trustworthy software-enabled things. In: 2020 IEEE Systems Security Symposium (SSS), pp 1–4, DOI 10.1109/SSS47320.2020.9174365

Mirakhorli M, Garcia D, Dillon S, Laporte K, Morrison M, Lu H, Koscinski V, Enoch C (2024) A landscape study of open source and proprietary tools for software bill of materials (sbom). arXiv preprint arXiv:240211151

Muirí ÉÓ (2019) Framing software component transparency: Establishing a common software bill of material (sbom). vol 12

Nocera S, Romano S, Di Penta M, Francese R, Scanniello G (2023) Software bill of materials adoption: A mining study from github. In: 2023 IEEE International Conference on Software Maintenance and Evolution (ICSME), IEEE, pp 39–49

NTIA (2021a) Ntia software component transparency. `https://www.ntia.gov/other-publication/ntia-software-component-transparency`, [SBOM Virtual Multistakeholder Meeting]

NTIA (2021b) Survey of existing sbom formats and standards. `https://ntia.gov/sites/default/files/publications/sbom_formats_survey-version-2021_0.pdf`, [SBOM Formats and Standards]

OWASP (2021) Cyclonedx. https://cyclonedxorg/

OWASP (2023a) Official cyclonedx specification documentation json reference v1.4. `https://cyclonedx.org/docs/1.4/json/`, [CycloneDx Documentation]

OWASP (2023b) Official cyclonedx tool center. `https://cyclonedx.org/tool-center/`, [CycloneDx Tools]

Ray B, Posnett D, Devanbu P, Filkov V (2017) A large-scale study of programming languages and code quality in github. Commun ACM 60(10):91–100, DOI 10.1145/3126905, URL `https://doi.org/10.1145/3126905`

Remaley EL, Remaley D (2021) Ntia sbom effort is progressing well. NTIA

Sanfilippo S (2023) Redis — redis.io. `https://redis.io/`, [Redis Library]

SPDX (2023a) Official SPDX Open Source Tools. `https://spdx.dev/tools-community/`, [Open Source SPDX Tools]

SPDX (2023b) Official SPDX Proprietary Tools. https://spdx.dev/tools-commercial/, [Commercial Proprietary SPDX Tools]
SPDX (2023c) Official SPDX Specification Documentation v2.3. https://spdx.github.io/spdx-spec/v2.3/package-information/, [SPDX Documentation]
van der Storm T (2007) Component-based configuration, integration and delivery
Storm VD, Tijs, et al. (2007) Component-based configuration, integration and delivery
Telecommunications N, (NTIA) IA (2021a) Minimum Elements for a Software Bill of Materials (SBOM). URL https://www.ntia.doc.gov/files/ntia/publications/sbom_minimum_elements_report.pdf, [NTIA2021]
Telecommunications N, (NTIA) IA (2021b) Sbom tool classification taxonomy. URL https://www.ntia.gov/files/ntia/publications/ntia_sbom_tooling_taxonomy-2021mar30.pdf, [NTIATaxonomy]
Tomczak M, Tomczak E (2014) The need to report effect size estimates revisited. an overview of some recommended measures of effect size. Trends in Sport Sciences
Waltermire DA, Feldman L, Witte GA (2016) Improving security and software management through the use of swid tags. David A. Waltermire, Larry Feldman, Gregory A. Witte
Wan Z, Xia X, Lo D, Murphy GC (2019) How does machine learning change software development practices? IEEE Transactions on Software Engineering 47(9):1857–1871
Wilcoxon F, Katti S, Wilcox RA, et al. (1970) Critical values and probability levels for the wilcoxon rank sum test and the wilcoxon signed rank test. Selected tables in mathematical statistics 1:171–259
Worthington J (2023) The world lags with sbom requirements, but likely not for long. Forrester
Xia B, Bi T, Xing Z, Lu Q, Zhu L (2023a) An empirical study on software bill of materials: Where we stand and the road ahead. In: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), IEEE, pp 2630–2642
Xia B, Zhang D, Liu Y, Lu Q, Xing Z, Zhu L (2023b) Trust in software supply chains: Blockchain-enabled sbom and the aibom future. arXiv preprint arXiv:230702088
Zerouali A, Mens T, Robles G, Gonzalez-Barahona JM (2019) On the diversity of software package popularity metrics: An empirical study of npm. In: 2019 IEEE 26th international conference on software analysis, Evolution and Reengineering (SANER), IEEE, pp 589–593
Zimmermann T (2016) Card-sorting: From text to themes. In: Perspectives on data science for software engineering, Elsevier, pp 137–141

A Appendix

Table 9: SPDXand CycloneDXTools, and their corresponding initialization code. We use t code patterns to identify Github projects that leverage these tools.

Tool Name	Code pattern to initialize SBOM tool in a project
bom	kubernetes-sigs/bom, bom generate
ort	oss-review-toolkit/ort
protobom	bom-squad/protobom/pkg/sbom, bom-squad/protobom
sbom-tool	microsoft/sbom-tool/releases, sbom-tool generate
spdx-maven-plugin	<artifactId>spdx-maven-plugin</artifactId> createSPDX
spdx-sbomgenerator	opensbom-generator/spdx-sbom-generator
Spdx-Java-Library	<artifactId>java-spdx-library</artifactId> Spdx-Java-Library
tools-golang	spdx/tools-golang
tools-java	tools-java-1.1.5-jar-with-dependencies.jar, /tools-java
tools-python	dxtools -i, spdx_tools.spdx.model, pypi/spdx-tools
cyclonedx-cli	CycloneDX/cyclonedx-cli
cyclonedx-conan	CycloneDX/cyclonedx-conan
cyclonedx-core-java	<artifactId>cyclonedx-core-java</artifactId>
cyclonedx-dotnet	cyclonedx-dotnet-generate, cyclonedx-dotnet
cyclonedx-dotnet-library	CycloneDX.Core, CycloneDX.Utils, CycloneDX.Spdx, CycloneDX.Spdx.Interop
cyclonedx-gomod	cyclonedx-gomod, CycloneDX/cyclonedx-gomod
cyclonedx-go	cyclonedx-go
cyclonedx-javascript-library	require('@cyclonedx/cyclonedx-library'), .Enums.ComponentType.Application
cyclonedx-maven-plugin	CycloneDX/cyclonedx-maven-plugin, cyclonedx-maven-plugin
cyclonedx-node-module	npx cyclonedx-bom, cyclonedx-node-module
cyclonedx-node-npm	cyclonedx-npm
cyclonedx-node-pnpm	cyclonedx-node-pnpm
cyclonedx-php-composer	cyclonedx/cyclonedx-php-composer
cyclonedx-php-library	CycloneDX/cyclonedx-php-library
cyclonedx-python-lib	CycloneDX/cyclonedx-python-lib
cyclonedx-ruby-gem	cyclonedx-ruby-gem, cyclonedx-ruby
cyclonedx-rust-cargo	CycloneDX/cyclonedx-rust-cargo
cyclonedx-webpack-plugin	new CycloneDxWebpackPlugin
cdxgen	cdxgen -t, .verify(bomSignature), npm:@cyclonedx/cdxgen

Continued on next page

Table 10 – continued from previous page

Name	Opensource	Supports Both Standards	NTIA Tool Categories								
			Produce			Consume			Transform		
			Build	Analyze	Edit	View	Diff	Import	Translate	Merge	Support

Table 10: Manual classification of 170 SBOM tools in NTIA's tool taxonomy.

Name	Opensource	Supports Both Standards	NTIA Tool Taxonomy use case								
			Produce			Consume			Transform		
			Build	Analyze	Edit	View	Diff	Import	Translate	Merge	Support
action-owasp-dependecy-track-check	✓	✗	✓	✓	✗	✓	✗	✗	✗	✗	✗
apt2sbom	✓	✓	✓	✗	✗	✗	✗	✗	✗	✗	✗
Auditjs	✓	✗	✓	✓	✗	✗	✗	✗	✗	✗	✓
Augur	✓	✗	✓	✓	✗	✗	✗	✗	✗	✗	✗
bom	✓	✗	✓	✓	✗	✓	✗	✗	✗	✗	✓
Cavil	✓	✗	✗	✗	✗	✓	✗	✗	✗	✗	✗
cdxgen	✓	✗	✓	✓	✗	✗	✗	✗	✗	✗	✓
Checkov	✓	✗	✓	✓	✗	✗	✗	✗	✗	✗	✗
CodeNotary CAS	✓	✓	✓	✓	✓	✓	✗	✗	✗	✗	✗
Codenotary CAS Authenticate Docker Image and SBOM	✓	✓	✓	✗	✗	✓	✗	✗	✗	✗	✗
Codenotary CAS Notarize Docker Image and SBOM	✓	✓	✓	✗	✗	✓	✗	✗	✗	✗	✗
Cosign	✓	✓	✓	✗	✗	✗	✗	✗	✗	✗	✗
Covenant	✓	✓	✓	✗	✗	✓	✗	✗	✓	✗	✓
cve-bin-tool	✓	✓	✓	✓	✗	✓	✗	✗	✗	✗	✓
CycloneDX .NET Generate SBOM	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗	✓
CycloneDX CLI	✓	✓	✓	✓	✓	✓	✓	✗	✓	✓	✗
CycloneDX Core for Java	✓	✗	✓	✓	✗	✗	✗	✓	✗	✗	✓
CycloneDX for .NET	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗	✗
CycloneDX for Erlang/Elixir (Mix)	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗	✓
CycloneDX for Erlang/Elixir (Rebar3)	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗	✓
CycloneDX for Go	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗	✓
CycloneDX for Gradle	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗	✓
CycloneDX for Maven	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗	✓
CycloneDX for Node.js	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗	✓
CycloneDX for NPM	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗	✓
CycloneDX for SBT (Scala)	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗	✓
CycloneDX for Webpack	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗	✓

Continued on next page

Table 10 – continued from previous page

			NTIA Tool Categories								
			Produce			Consume			Transform		
Name	Opensource	Supports Both Standards	Build	Analyze	Edit	View	Diff	Import	Translate	Merge	Support
CycloneDX JavaScript Library	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗	✓
CycloneDX Libraries for .NET	✓	✗	✓	✗	✗	✗	✗	✓	✗	✗	✓
CycloneDX library for Go	✓	✗	✓	✗	✗	✗	✗	✓	✗	✗	✗
CycloneDX PHP Composer Generate SBOM	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗	✓
CycloneDX PHP Library	✓	✗	✓	✗	✓	✗	✗	✗	✓	✗	✗
CycloneDX Python Generate SBOM	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗	✓
CycloneDX Python Library	✓	✗	✓	✗	✗	✓	✓	✓	✓	✗	✓
CycloneDX Rust	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓
CycloneDX Web Tool	✓	✗	✗	✗	✗	✓	✗	✗	✓	✓	✗
cyclonedx-merge	✓	✗	✗	✗	✗	✗	✗	✗	✗	✓	✗
cyclonedx-npm-pipe	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗	✓
DaggerBoard	✓	✓	✗	✓	✗	✓	✗	✓	✗	✗	✗
Dependency-Track Maven Plugin	✓	✗	✓	✓	✗	✓	✗	✓	✗	✗	✓
Distro2SBOM	✓	✓	✓	✗	✗	✗	✗	✗	✗	✗	✓
docker-sbom-cli-plugin	✓	✓	✓	✗	✗	✗	✗	✗	✗	✗	✗
EMBA	✓	✗	✓	✓	✗	✓	✗	✗	✗	✗	✗
FOSSLight	✓	✗	✗	✓	✓	✓	✗	✗	✓	✓	✗
FOSSology	✓	✓	✗	✓	✓	✓	✓	✗	✓	✓	✓
Generate SBoM for Elixir project	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗	✓
gh-sbom	✓	✓	✓	✗	✗	✗	✗	✗	✗	✗	✓
GitHub Self-Service SBOMs	✓	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗
gobom	✓	✗	✓	✗	✗	✗	✗	✗	✗	✓	✗
Grype	✓	✓	✓	✗	✗	✓	✗	✓	✗	✗	✗
in-toto	✓	✓	✓	✗	✗	✗	✗	✗	✗	✗	✗
Jake	✓	✗	✓	✗	✗	✓	✗	✗	✗	✗	✗
jbom	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗	✗
kbom - Kubernetes Bill of Materials powered by KSOC	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗	✗
KubeClarity	✓	✓	✓	✓	✗	✓	✗	✗	✗	✓	✓
Kyverno	✓	✗	✓	✓	✗	✓	✗	✗	✗	✗	✓
Lagoon Insights Handler	✓	✓	✗	✗	✗	✗	✗	✓	✗	✗	✓
Lib4sbom	✓	✓	✓	✗	✓	✗	✗	✓	✗	✗	✗
LicenseComplianceTool	✓	✗	✗	✗	✓	✗	✗	✓	✗	✗	✗
macaron	✓	✗	✗	✗	✗	✗	✗	✓	✗	✗	✓
mdbom	✓	✗	✗	✗	✗	✓	✗	✗	✓	✗	✗
MLBOMdoc	✓	✗	✗	✓	✗	✓	✗	✓	✗	✗	✗
Nexus Lifecycle Jenkins Plugin	✓	✗	✓	✓	✗	✓	✗	✗	✗	✗	✗

Continued on next page

Table 10 – continued from previous page

			NTIA Tool Categories								
			Produce			Consume			Transform		
Name	Opensource	Supports Both Standards	Build	Analyze	Edit	View	Diff	Import	Translate	Merge	Support
Nix / Nixpkgs	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗	✗
ntia-conformance-checker	✓	✗	✗	✓	✗	✗	✗	✓	✗	✗	✓
OSS Review Toolkit (ORT)	✓	✓	✓	✓	✗	✓	✓	✓	✗	✗	✗
OSS Inventory	✓	✗	✗	✓	✗	✓	✗	✓	✗	✗	✗
Parlay	✓	✓	✗	✗	✓	✗	✗	✗	✗	✗	✗
Protobom	✓	✓	✓	✗	✗	✗	✗	✓	✓	✗	✗
REuse	✓	✗	✓	✓	✗	✗	✗	✗	✗	✗	✗
Salus	✓	✗	✓	✓	✗	✓	✗	✗	✗	✗	✓
SBOM Assembler	✓	✓	✗	✗	✗	✗	✗	✓	✗	✓	✗
SBOM CLI	✓	✓	✓	✗	✗	✗	✗	✗	✗	✗	✗
SBOM Explorer	✓	✓	✗	✗	✗	✗	✗	✓	✗	✗	✗
SBOM Grep	✓	✓	✗	✓	✗	✓	✗	✓	✗	✗	✗
SBOM-Manager	✓	✓	✗	✓	✗	✗	✗	✓	✗	✗	✗
sbom-submission-action	✓	✗	✗	✓	✗	✗	✗	✗	✗	✗	✓
sbom-swissarmy-bitbucket-pipe	✓	✓	✓	✓	✗	✗	✗	✗	✗	✗	✓
sbom-tool	✓	✗	✓	✓	✗	✗	✗	✗	✗	✗	✓
SBOM4Files	✓	✓	✓	✗	✗	✗	✗	✗	✗	✗	✓
SBOMAudit	✓	✓	✗	✓	✗	✓	✗	✓	✗	✗	✗
sbomqs	✓	✓	✗	✓	✗	✓	✗	✓	✗	✗	✗
SBOMTrend	✓	✓	✗	✓	✗	✗	✓	✗	✗	✗	✗
sca-codeinsight-reports-cyclonedx	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗	✗
Scancode Toolkit	✓	✓	✗	✓	✗	✓	✗	✗	✗	✗	✗
SecObserve	✓	✗	✓	✓	✗	✓	✗	✗	✗	✗	✓
Semgrep	✓	✗	✓	✓	✗	✓	✗	✗	✗	✗	✗
SnykVulnCheck	✓	✗	✗	✓	✗	✗	✗	✗	✗	✗	✗
SPDX Golang Libraries	✓	✗	✓	✓	✗	✓	✗	✓	✓	✗	✓
SPDX Java Libraries and Tools	✓	✗	✓	✓	✗	✓	✓	✓	✓	✓	✓
SPDX JavaScript Libraries	✓	✗	✗	✓	✗	✗	✓	✗	✓	✗	✓
SPDX Maven Plugin	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗	✓
SPDX Online Tools	✓	✗	✗	✓	✓	✓	✓	✓	✓	✗	✓
SPDX Python Libraries	✓	✗	✓	✓	✗	✗	✗	✓	✓	✗	✓
spdx-sbom-generator	✓	✗	✓	✓	✗	✗	✗	✗	✗	✗	✓
spdxcyclone	✓	✓	✗	✗	✗	✗	✗	✗	✓	✗	✗
SRC:CLR SBOM Generator	✓	✗	✓	✓	✗	✗	✗	✗	✗	✗	✗
SW360	✓	✗	✗	✗	✗	✓	✓	✓	✗	✓	✗
SwiftBOM	✓	✓	✓	✗	✗	✓	✗	✗	✗	✗	✗
Syft	✓	✓	✓	✗	✗	✗	✗	✗	✓	✗	✓
Tern	✓	✓	✓	✓	✗	✓	✗	✗	✗	✗	✓
ThreatMapper	✓	✓	✓	✓	✗	✓	✗	✗	✗	✗	✗
Valaa Stack	✓	✗	✓	✗	✗	✗	✗	✗	✓	✗	✗

Continued on next page

Table 10 – continued from previous page

			NTIA Tool Categories								
			Produce			Consume			Transform		
Name	Opensource	Supports Both Standards	Build	Analyze	Edit	View	Diff	Import	Translate	Merge	Support
Vexy	✓	✗	✓	✓	✗	✗	✗	✗	✗	✗	✗
vsm-sbom-booster	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗	✓
Vuls	✓	✗	✓	✓	✗	✓	✗	✗	✗	✗	✗
yasca (Yet Another SCA tool)	✓	✗	✓	✗	✗	✓	✗	✗	✗	✗	✗
Yocto Project / OpenEmbedded	✓	✗	✓	✓	✗	✗	✗	✗	✗	✗	✗
Apiiro	✗	✗	✓	✓	✗	✓	✗	✗	✗	✗	✓
Arnica	✗	✗	✓	✓	✗	✓	✗	✗	✗	✗	✓
Arsenal	✗	✗	✓	✓	✗	✓	✗	✓	✗	✗	✗
Athena	✗	✗	✓	✓	✗	✓	✗	✗	✗	✗	✗
Beniva Software Bill of Materials (SBOM)	✗	✗	✗	✓	✗	✓	✗	✓	✗	✗	✗
Black Duck	✗	✓	✓	✓	✗	✓	✗	✓	✗	✗	✓
BlackBerry Jarvis	✗	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗
Bytesafe	✗	✓	✗	✓	✗	✓	✗	✓	✗	✗	✓
CAST Highlight	✗	✗	✓	✓	✗	✓	✗	✓	✗	✗	✓
Codenotary vcn	✗	✓	✓	✓	✗	✓	✗	✗	✗	✗	✗
CodeSentry	✗	✗	✓	✓	✗	✓	✗	✗	✗	✗	✓
Contrast Security	✗	✓	✓	✓	✗	✓	✗	✗	✗	✗	✗
CxSCA	✗	✗	✓	✓	✗	✓	✗	✗	✗	✗	✓
Cybeats SBOM Studio	✗	✓	✓	✓	✗	✗	✗	✓	✓	✓	✓
Cybellum SBOM	✗	✗	✓	✓	✓	✓	✗	✓	✓	✓	✓
Debricked	✗	✗	✓	✓	✗	✓	✗	✗	✗	✗	✓
DejaCode	✗	✗	✓	✓	✓	✓	✗	✗	✓	✗	✗
Endor Labs	✗	✓	✓	✓	✗	✓	✗	✗	✗	✗	✗
Enso	✗	✓	✓	✓	✗	✓	✗	✗	✗	✗	✓
FACT	✗	✓	✓	✓	✗	✓	✗	✗	✗	✗	✗
Flawnter	✗	✓	✓	✓	✗	✗	✗	✗	✗	✗	✓
Fortify on Demand	✗	✗	✓	✓	✗	✓	✗	✗	✗	✗	✗
Fortress File Integrity Assurance	✗	✓	✓	✓	✗	✓	✓	✓	✓	✗	✓
FOSSA	✗	✓	✓	✓	✗	✓	✗	✓	✗	✗	✗
FOSSID	✗	✓	✓	✓	✗	✓	✓	✓	✗	✗	✗
Heimdall	✗	✓	✓	✓	✗	✓	✗	✓	✓	✗	✗
Interlynk	✗	✓	✓	✓	✗	✓	✗	✓	✓	✓	✗
Ion Channel Platform	✗	✓	✓	✓	✗	✓	✗	✓	✗	✗	✗
JDisc Discovery	✗	✗	✓	✓	✗	✓	✗	✓	✗	✗	✓
JupiterOne	✗	✗	✓	✓	✗	✓	✗	✗	✗	✗	✗
Kondukto	✗	✓	✓	✓	✗	✓	✗	✓	✗	✗	✓
Manifest	✗	✓	✓	✓	✗	✓	✓	✓	✗	✓	✓
MedScan	✗	✓	✗	✓	✗	✗	✗	✓	✗	✗	✗
Mend SCA	✗	✓	✓	✓	✗	✓	✗	✗	✗	✗	✗
NetRise Turbine	✗	✓	✓	✓	✗	✓	✗	✓	✗	✓	✗
Nexus IQ	✗	✓	✓	✓	✗	✓	✗	✗	✗	✗	✓
NowSecure Platform	✗	✗	✓	✓	✗	✓	✗	✗	✗	✗	✗
ONEKEY firmware analysis platform	✗	✓	✓	✓	✗	✓	✗	✗	✗	✗	✓
Prisma Cloud	✗	✗	✓	✓	✗	✓	✗	✗	✗	✗	✗

Continued on next page

Table 10 – continued from previous page

			NTIA Tool Categories								
			Produce			Consume			Transform		
Name	Opensource	Supports Both Standards	Build	Analyze	Edit	View	Diff	Import	Translate	Merge	Support
PulseUno Plugin for Dimensions CM	✗	✗	✓	✓	✗	✓	✗	✗	✗	✗	✗
Reliza Hub	✗	✗	✓	✗	✗	✓	✗	✓	✗	✗	✓
Rezilion Dynamic SBOM	✗	✓	✓	✓	✗	✓	✗	✗	✗	✗	✓
RKVST SBOM Hub	✗	✓	✓	✓	✗	✗	✗	✗	✗	✗	✗
SBOM Benchmark	✗	✓	✗	✓	✗	✓	✓	✗	✗	✗	✗
SBOM Insights	✗	✓	✓	✓	✓	✓	✗	✓	✗	✓	✓
SBOM Observer	✗	✓	✓	✓	✗	✓	✗	✓	✗	✗	✓
SBOM.sh	✗	✓	✓	✗	✗	✓	✗	✓	✗	✗	✗
SCANOSS	✗	✓	✓	✓	✗	✓	✗	✓	✗	✗	✓
SecureStack	✗	✗	✓	✓	✗	✗	✗	✗	✗	✗	✓
Snyk	✗	✓	✓	✓	✗	✓	✗	✓	✗	✗	✗
Software Assurance Guardian Point Man	✗	✓	✓	✓	✗	✓	✗	✓	✗	✗	✗
Sonatype Lift	✗	✓	✓	✓	✗	✓	✗	✗	✗	✗	✗
SourceAuditor	✗	✗	✗	✓	✗	✓	✓	✓	✓	✗	✗
StackAware	✗	✗	✗	✓	✗	✓	✗	✗	✗	✗	✗
Tidelift	✗	✓	✓	✓	✗	✓	✗	✓	✗	✗	✗
TrustSource	✗	✓	✗	✓	✓	✓	✗	✓	✗	✗	✓
Veracode	✗	✗	✓	✓	✗	✗	✗	✗	✗	✗	✗
Vigilant Ops	✗	✓	✓	✓	✗	✓	✗	✗	✓	✗	✓
Vulert	✗	✓	✗	✓	✗	✓	✗	✓	✗	✗	✗
Vulnerabilities.io	✗	✓	✓	✓	✗	✓	✗	✗	✗	✗	✓
Xray	✗	✓	✓	✓	✗	✓	✗	✗	✗	✗	✗
Xygeni Software Supply-Chain Security	✗	✓	✓	✗	✗	✓	✗	✗	✗	✗	✓